

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos



Grupo 1

Ing. Angel Isidro Mercado

Tarea 1. Sistemas Operativos

320304435 - Cabrera Rojas Oscar

Ciudad Universitaria, CDMX a 25 de octubre del 2026

Índice

1. Fundamentos del software y los sistemas operativos **1**
- 1.1. Naturaleza y clasificación del software . . . 1
- 1.2. Definición y arquitectura del SO . . . 2
- 1.3. Funciones y características de los SO . . . 3
- 1.4. Evolución de los SO . . . 4

2. Catálogo de 120 sistemas operativos **6**
- 2.1. Criterios de clasificación . . . 6
- 2.2. Listado de sistemas operativos por familias . . . 7

3. Sistemas operativos actuales y para bajos recursos **10**
- 3.1. Diez sistemas operativos actuales . . . 10
- 3.2. Diez sistemas operativos para bajos recursos . . . 12

4. Virtualización de sistemas operativos **14**
- 4.1. Conceptos e hipervisores . . . 14
- 4.2. Software de virtualización . . . 15

5. Procesos en C:\Windows\System32 **16**
- 5.1. Arquitectura y subsistemas de ejecución en System32 . . . 16
- 5.2. Procesos críticos del sistema y del núcleo . . . 17
- 5.3. Utilerías ejecutables .com . . . 18
- 5.4. Catálogo de programas y herramientas de System32 . . . 19
  - 5.4.1. Almacenamiento, sistemas de archivos y particiones . . . 19
  - 5.4.2. Redes, comunicaciones y diagnóstico . . . 20
  - 5.4.3. Gestión de procesos, rendimiento y supervisión . . . 21
  - 5.4.4. Mantenimiento, imágenes, arranque y energía . . . 22
  - 5.4.5. Seguridad, control de acceso y criptografía . . . 22
  - 5.4.6. Registro de Windows, servicios y motores de script . . . 23
  - 5.4.7. Filtrado de texto, datos y utilerías de soporte . . . 23
  - 5.4.8. Consolas de gestión Microsoft (MMC y complementos .msc) . . . 24

6. Administración de recursos y comandos **24**
- 6.1. Administración de memoria . . . 24
  - 6.1.1. Esquemas de asignación contigua y fragmentación . . . 25
  - 6.1.2. Paginación pura, segmentación y memoria virtual . . . 26
  - 6.1.3. Políticas de reemplazo de páginas e hiperpaginación . . . 26
- 6.2. Administración de procesos . . . 27
  - 6.2.1. Bloque de Control del Proceso (PCB) y modelo de estados . . . 27
  - 6.2.2. Hilos de ejecución y planificación de CPU . . . 28
  - 6.2.3. Sincronización y comunicación interproceso (IPC) . . . 29
- 6.3. Comandos de sistemas operativos . . . 29
  - 6.3.1. Navegación y manipulación de archivos y directorios . . . 30
  - 6.3.2. Gestión, supervisión y control de procesos . . . 30
  - 6.3.3. Redes, conectividad y telecomunicaciones . . . 31
  - 6.3.4. Permisos, usuarios y control de acceso . . . 31
  - 6.3.5. Búsqueda, filtrado y procesamiento de flujos de texto . . . 32

6.3.6. Almacenamiento, sistemas de archivos y montaje 32
6.3.7. Administración del sistema, servicios y control de energía 32

7. Conclusiones personales **33**

Referencias **34**

1. Fundamentos del software y los sistemas operativos

1.1. Naturaleza y clasificación del software

En la ciencia de la computación, un sistema informático integral se fundamenta en la interacción continua entre dos entidades complementarias: la infraestructura física o *hardware* y los componentes lógicos e intangibles denominados *software*. De acuerdo con la taxonomía clásica de la ingeniería en computación, el software comprende la totalidad de programas, procedimientos, reglas, rutinas y documentación asociada que instruyen a la circuitería electrónica para llevar a cabo secuencias ordenadas de procesamiento de datos (McHoes & Flynn, 2011, §1.1, p. 3). Sin el soporte de este conjunto de instrucciones, la máquina permanece en un estado inoperante, comúnmente descrito como «máquina desnuda» (*bare machine*), incapaz de ejecutar tareas estructuradas por sí misma (Carretero Pérez et al., 2001, §2.1, p. 55).

Para dimensionar la diversidad y jerarquía del software dentro de un sistema moderno, se adopta de manera generalizada una clasificación tripartita basada en el nivel de proximidad con el hardware y la finalidad operativa de cada programa:

1. **Software de sistema:** Agrupa al conjunto de programas encargados de gestionar, coordinar y controlar directamente los recursos físicos de la computadora (procesador, memoria, dispositivos periféricos y buses de comunicación), proporcionando al mismo tiempo una plataforma de abstracción homogénea sobre la cual operan los demás programas. En esta categoría se ubican:
    - El *sistema operativo* (SO), como núcleo rector del sistema.
    - Los controladores de dispositivos (*device drivers*), que traducen comandos abstractos en secuencias de señales electrónicas para cada periférico.
    - Los programas de arranque y cargadores (*loaders*), que inicializan la memoria durante el encendido.
    - Las utilerías del sistema, que efectúan tareas de diagnóstico, desfragmentación, monitoreo y mantenimiento del almacenamiento.

2. **Software de programación o desarrollo:** Comprende las herramientas computacionales creadas específicamente para asistir al desarrollador e ingeniero de software en la concepción, escritura, traducción, depuración y mantenimiento de nuevos programas. Entre sus componentes esenciales se destacan:
    - Traductores de lenguaje: compiladores (que generan código objeto autónomo) e intérpretes (que ejecutan código fuente de forma directa mediante análisis dinámico).
    - Ensambladores, que traducen instrucciones mnemónicas a lenguaje máquina binario.
    - Enlazadores (*linkers*), que combinan módulos objeto independientes y bibliotecas precompiladas en un único archivo ejecutable.
    - Depuradores (*debuggers*), perfiles de ejecución (*profilers*) y entornos de desarrollo integrado (IDE).

3. **Software de aplicación:** Representa el estrato superior del sistema, diseñado directamente para resolver necesidades específicas y productivas de los usuarios finales, ya sean de índole científica, corporativa, educativa o recreativa (Silberschatz et al., 2006, §1.1, p. 4). Ejemplos representativos incluyen:

- Sistemas de gestión de bases de datos relacionales y distribuidas.
- Procesadores de texto, hojas de cálculo y suites ofimáticas.
- Navegadores web y clientes de comunicaciones distribuidas.
- Paquetes de diseño asistido por computadora (CAD) y software de simulación numérica.

1.2. Definición y arquitectura del SO

El sistema operativo (SO) constituye la pieza medular del software de sistema. Desde una perspectiva formal, se define como un programa maestro que actúa como intermediario entre los programas de aplicación y el hardware de la computadora, gobernando la asignación de recursos y ofreciendo a los programadores una interfaz uniforme que simplifica el control de la máquina (Stallings, 2018, §2.1, p. 69).

En la literatura especializada se reconocen dos perspectivas complementarias para definir el rol de un SO (Silberschatz et al., 2006, §1.1, p. 4):

- **Vista del usuario (máquina extendida o virtual):** Para el usuario y el desarrollador de aplicaciones, el hardware en su estado físico resulta hostil y excesivamente complejo; la manipulación directa de registros de entrada/salida (E/S), interrupciones vectorizadas y sincronización de buses exigiría un esfuerzo desmedido. El SO cubre estas complejidades ofreciendo una «máquina extendida» o máquina virtual, caracterizada por abstracciones intuitivas de alto nivel (archivos, procesos, zócalos de red y ventanas gráficas).

- **Vista del sistema (asignador de recursos y programa de control):** Desde el punto de vista arquitectónico interno, el SO opera como un riguroso administrador y asignador de recursos (*resource allocator*). Su responsabilidad estriba en arbitrar las demandas concurrentes y en ocasiones conflictivas de la unidad central de procesamiento (CPU), la memoria principal y los dispositivos periféricos. En este papel, actúa como un programa de control que supervisa permanentemente la ejecución de los programas para prevenir accesos indebidos, colisiones de memoria y bloqueos del sistema (Deitel et al., 2004, §1.3, p. 7).

Para garantizar la estabilidad y la seguridad del sistema frente a instrucciones potencialmente peligrosas, los procesadores modernos incorporan al menos dos modos de ejecución diferenciados por hardware: el *modo usuario* (*user mode*), en el que se restringe el acceso directo a la memoria física y a las instrucciones privilegiadas de E/S, y el *modo núcleo* o modo supervisor (*kernel mode*), en el cual el núcleo del SO posee privilegios absolutos sobre las estructuras del procesador. La transición entre ambos modos se ejecuta de forma controlada mediante interrupciones de software o instrucciones trampa (*trap*), las cuales transfieren el flujo de ejecución hacia las rutinas del núcleo a través de llamadas al sistema (*system calls*).

A nivel de organización interna, los sistemas operativos han adoptado diversos paradigmas arquitectónicos a lo largo de su evolución (Deitel et al., 2004, §1.13, p. 28):

- **Arquitectura monolítica:** Todos los servicios fundamentales (planificador de la CPU, gestión de memoria, sistemas de archivos, pila de red y controladores de dispositivos) se compilan juntos y residen en el mismo espacio de direcciones privilegiado en memoria. Esta arquitectura destaca por su extraordinario rendimiento y baja latencia en la comunicación interna, aunque presenta una alta fragilidad: cualquier fallo crítico o desbordamiento en un controlador de dispositivo puede corromper el núcleo entero y provocar

una caída completa del sistema (*kernel panic*). Linux y los núcleos clásicos de UNIX representan los mayores exponentes de este enfoque.

- **Arquitectura por capas:** El sistema se estructura en una jerarquía estricta de estratos superpuestos, donde cada capa N ofrece una interfaz de servicios a la capa inmediatamente superior ($N + 1$) valiéndose únicamente de los servicios provistos por la capa inferior ($N - 1$). Si bien aporta gran claridad en el diseño, depuración y verificación formal, penaliza el rendimiento por la sobrecarga que impone el paso secuencial de mensajes entre múltiples capas.
- **Arquitectura de micronúcleo (*microkernel*):** Propone reducir el código del núcleo al mínimo indispensable para asegurar el funcionamiento de la máquina: gestión básica de espacios de memoria, sincronización elemental, despacho de hilos y un mecanismo de comunicación interprocesos (IPC). Todos los demás servicios tradicionales (controladores, sistemas de archivos, subsistemas de red y servidores de políticas) se trasladan al espacio de usuario en calidad de procesos servidores independientes. Este modelo otorga máxima tolerancia a fallos y modularidad, dado que el colapso de un controlador no afecta al resto del sistema, aunque introduce una penalización en el rendimiento derivada del intercambio masivo de mensajes entre contextos de usuario. Mach, Minix y QNX son ejemplos emblemáticos.
- **Arquitectura híbrida:** Combina la organización modular de los micronúcleos con la eficiencia de los núcleos monolíticos, ejecutando ciertos componentes críticos (como subsistemas gráficos o pilas de red) dentro del espacio de direcciones del núcleo para mitigar la sobrecarga de conmutación de contexto. Esta filosofía rige la arquitectura de Microsoft Windows NT y el núcleo XNU de macOS.

1.3. Funciones y características de los SO

El funcionamiento armónico de una computadora requiere que el SO desempeñe de forma concurrente un conjunto de funciones críticas e interrelacionadas (Stallings, 2018, §2.1, p. 70):

1. **Administración de procesos y planificación de la CPU:** El SO es responsable de crear, suspender, reanudar y finalizar procesos e hilos de ejecución. Mediante el planificador de la CPU, asigna el tiempo de procesamiento entre las tareas listas para ejecutarse conforme a políticas que buscan maximizar el rendimiento, minimizar el tiempo de respuesta y garantizar equidad. Asimismo, provee mecanismos para la sincronización y la comunicación ordenada entre procesos concurrentes.
2. **Administración de memoria principal:** Supervisa el mapa de ocupación de la memoria primaria (RAM), decidiendo qué programas se cargan, qué espacio se les asigna y cuándo debe liberarse. A través del mecanismo de memoria virtual, aísla los espacios de direcciones lógicas de cada proceso mediante paginación y segmentación, evitando que un programa altere el espacio de otro o acceda indebidamente al núcleo.
3. **Administración del almacenamiento secundario y sistemas de archivos:** Abstrae las propiedades físicas de discos magnéticos, unidades de estado sólido (SSD) y medios ópticos en estructuras lógicas comprensibles denominadas archivos y directorios. Gobierna la asignación de bloques, la consistencia de los metadatos, el espacio libre y los mecanismos de control de acceso y permisos.

4. **Administración de entrada/salida (E/S):** Oculta las peculiaridades mecánicas y electrónicas de los dispositivos periféricos a través de una capa común de manejadores (*drivers*). Proporciona técnicas de almacenamiento temporal en memoria intermedia (*buffering*), retención rápida en memoria caché (*caching*) y colocación de trabajos en colas virtuales de espera (*spooling*) para sincronizar dispositivos lentos con la velocidad del procesador.

5. **Seguridad y protección del sistema:** Garantiza que los recursos solo sean accedidos por usuarios y programas con los privilegios debidos. Implementa listas de control de acceso (ACL), autenticación de identidad, mecanismos de auditoría y separación estricta de privilegios.

6. **Suministro de interfaces:** Provee los canales de interacción tanto para seres humanos —a través de interfaces de línea de comandos (CLI) o interfaces gráficas de usuario (GUI)— como para los programas de software mediante la interfaz de programación de aplicaciones (API) y las llamadas al sistema.

Junto a estas funciones, los sistemas operativos modernos exhiben cinco características esenciales de diseño:

- **Concurrencia y multiprogramación:** Habilidad de mantener múltiples programas activos simultáneamente en memoria, alternando la ejecución de sus hilos de modo que la CPU nunca permanezca inactiva mientras existan tareas pendientes.

- **Compartición eficiente de recursos:** Distribución equitativa y transparente del hardware entre múltiples usuarios o aplicaciones concurrentes sin degradación catastrófica del servicio.

- **No determinismo y manejo de eventos asíncronos:** Capacidad de responder en tiempo real a estímulos externos e interrupciones impredecibles de hardware sin comprometer el estado interno del sistema.

- **Determinismo en la protección:** Certeza de que la ejecución aislada de un proceso erróneo o malicioso no alterará los datos de otros procesos ni corromperá el núcleo del sistema.

- **Evolucionabilidad:** Estructura modular concebida para admitir parches, actualizaciones, nuevos controladores de dispositivos y extensiones arquitectónicas sin obligar a rediseñar el sistema en su totalidad (Stallings, 2018, §2.1, p. 71).

1.4. Evolución de los SO

La historia de los sistemas operativos está íntimamente ligada a la evolución de las arquitecturas de hardware y a las demandas crecientes de automatización y eficiencia en el aprovechamiento de la computadora (Carretero Pérez et al., 2001, §2.13, p. 67). Esta trayectoria suele dividirse en cinco generaciones distintivas:

- **Primera generación (1945–1955) — Tubos de vacío y tableros de conmutación:** Las primeras computadoras electrónicas (como ENIAC o EDVAC) carecían por completo de sistema operativo. Los programadores reservaban bloques de tiempo directamente en la

consola de la máquina y cargaban sus programas codificados en lenguaje máquina absoluto mediante tableros enchufables o tarjetas perforadas. Si ocurría un error, el depurado requería inspeccionar manualmente los bancos de luces y conmutadores. Toda la gestión de entrada/salida y asignación era estrictamente manual.

- **Segunda generación (1955–1965) — Transistores y sistemas por lotes:** Con la introducción de los transistores aumentó la confiabilidad del hardware, pero la diferencia de velocidad entre la CPU y los dispositivos periféricos (lectores de tarjetas e impresoras) provocaba tiempos muertos inaceptables. Para amortizar el costo de las máquinas se diseñaron los primeros sistemas de procesamiento por lotes (*batch processing*) gobernados por monitores residentes (como FMS, *Fortran Monitor System*, e IBSYS en computadoras IBM 7094) (Stallings, 2018, §2.2, p. 72). El operador agrupaba trabajos similares en lotes sobre cinta magnética, y el monitor residente se encargaba de encadenar automáticamente la ejecución de un trabajo tras otro sin intervención manual.

- **Tercera generación (1965–1980) — Circuitos integrados, multiprogramación y tiempo compartido:** La tecnología de circuitos integrados posibilitó dos avances revolucionarios: la multiprogramación y el tiempo compartido (*time-sharing*). La multiprogramación permitió albergar varios programas en memoria a la vez; cuando el programa en curso solicitaba una operación de E/S, la CPU se conmutaba de inmediato hacia otro programa listo, multiplicando el coeficiente de utilización del procesador. El tiempo compartido llevó este principio un paso más allá al asignar a cada usuario interactivo un intervalo de tiempo (*quantum*), creando la ilusión de poseer la máquina en exclusiva. Hitos decisivos de esta etapa fueron el diseño de la familia IBM System/360 con OS/360, el desarrollo del sistema MULTICS en el MIT y el surgimiento en 1969 de UNIX en los laboratorios Bell de AT&T, concebido con principios de simplicidad, portabilidad y modularidad que perduran hasta la actualidad.

- **Cuarta generación (1980–1990) — Microprocesadores y computación personal:** La integración a gran escala (LSI y VLSI) permitió la fabricación masiva de microprocesadores y el advenimiento de las computadoras personales (PC) y estaciones de trabajo. Los sistemas operativos evolucionaron para atender a un usuario individual, priorizando la facilidad de uso mediante interfaces de línea de comandos altamente accesibles (como MS-DOS y CP/M) y, posteriormente, interfaces gráficas de usuario (GUI) basadas en ventanas, iconos y ratón, impulsadas por el Xerox Alto, el Apple Macintosh (Classic Mac OS) y Microsoft Windows. En paralelo, surgieron implementaciones comerciales de UNIX como SunOS/Solaris, HP-UX e IRIX, así como la variante académica BSD.

- **Quinta generación (1990–presente) — Cómputo distribuido, movilidad y nube:** Caracterizada por la ubicuidad de las redes de datos, arquitecturas de multiprocesamiento simétrico (SMP) y procesadores multinúcleo. Destaca el nacimiento y consolidación de Linux en 1991 por Linus Torvalds y el auge del software libre bajo la licencia GPL. La arquitectura Microsoft Windows NT sentó las bases de la línea profesional que culmina en Windows 10 y Windows 11. En el ámbito móvil, la miniaturización y la conectividad inalámbrica impulsaron el predominio global de Android (núcleo Linux) e iOS (núcleo Darwin/XNU). En la actualidad, la virtualización de servidores a nivel de hipervisores y la contenerización han convertido a los sistemas operativos en plataformas optimizadas para centros de datos masivos y entornos de computación en la nube.

Para sintetizar de manera gráfica y estructurada la evolución de las cinco generaciones descritas, la Imagen 1 presenta el cuadro sinóptico correspondiente a este recorrido tecnológico.

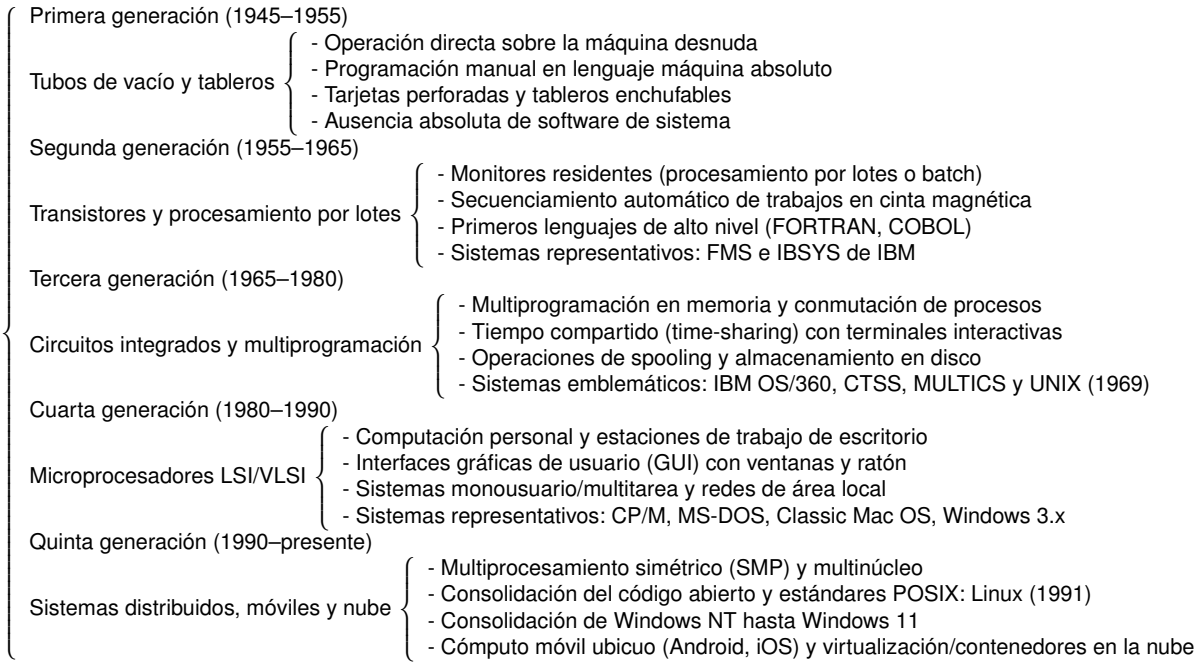


Imagen 1: Evolución histórica de las generaciones de sistemas operativos.

2. Catálogo de 120 sistemas operativos

2.1. Criterios de clasificación

El ecosistema de los sistemas operativos abarca más de seis décadas de desarrollo continuo, dando lugar a una vasta diversidad de plataformas concebidas para satisfacer requerimientos que van desde la computación centralizada en grandes computadoras (*mainframes*) hasta nodos embebidos con recursos severamente restringidos (Deitel et al., 2004, §1.3, p. 7). Con el propósito de caracterizar esta diversidad y evitar la redundancia entre revisiones menores de una misma plataforma, la conformación del catálogo responde a tres criterios metodológicos:

1. **Identidad arquitectónica unívoca:** Cada entrada corresponde a un proyecto, distribución o arquitectura de sistema operativo diferenciada por su árbol genealógico, su modelo de desarrollo o su propósito de diseño, descartando revisiones menores continuistas como entradas independientes.

2. **Especificación canónica de versión:** Cada sistema operativo se asocia con una versión formal canónica o hito de desarrollo representativo, señalando ediciones mayores, paquetes de servicio o versiones con soporte extendido.

3. **Estructuración por familias tecnológicas:** Los sistemas se distribuyen en ocho familias computacionales: sistemas históricos y de grandes computadoras; UNIX comercial y derivados BSD; distribuciones de la familia GNU/Linux; ecosistema Microsoft, MS-DOS

y afines; sistemas operativos en tiempo real (RTOS) y de control crítico; plataformas móviles, embebidas y vestibles; sistemas académicos, experimentales y de micronúcleo; y sistemas especializados de red, almacenamiento e infraestructura.

2.2. Listado de sistemas operativos por familias

A continuación se presenta la relación taxonómica de 120 sistemas operativos organizados por familias computacionales, especificando en cada caso su denominación técnica, versión representativa y ámbito operativo:

Familia 1: Sistemas históricos y mainframes

1. **IBM OS/360** — *Versión: Release 21.8*. Lotes y multiprogramación MVT en System/360.
2. **IBM z/OS** — *Versión: V2R5*. Mainframe corporativo de 64 bits en z/Architecture.
3. **IBM z/VM** — *Versión: 7.3*. Hipervisor de virtualización masiva en mainframes.
4. **IBM OS/400 (IBM i)** — *Versión: V7R5*. Plataforma integrada orientada a objetos en POWER.
5. **MULTICS** — *Versión: MR 12.6f*. Sistema pionero de tiempo compartido y seguridad anular.
6. **CTSS** — *Versión: Rev. 2.0*. Compatible Time-Sharing System pionero del MIT.
7. **DEC OpenVMS** — *Versión: V9.2-1*. Sistema de alta disponibilidad para VAX y x86-64.
8. **DEC TOPS-10** — *Versión: 7.04*. Sistema de tiempo compartido para arquitectura PDP-10.
9. **DEC TOPS-20** — *Versión: Release 7.1*. Memoria virtual interactiva en PDP-10/DEC-20.
10. **CDC NOS** — *Versión: 2.8.7*. Network Operating System para supercomputadoras Cyber.
11. **Univac EXEC 8** — *Versión: Level 38*. Sistema en tiempo real y por lotes en Univac 1100.
12. **Burroughs MCP** — *Versión: XVII*. Master Control Program pionero en memoria virtual.

Familia 2: UNIX comercial y derivados BSD

13. **AT&T UNIX System V** — *Versión: Release 4 (SVR4)*. Estándar comercial histórico.
14. **SunOS** — *Versión: 4.1.4*. Sistema UNIX basado en BSD para plataformas SPARC.
15. **Oracle Solaris** — *Versión: 11.4*. UNIX empresarial con soporte nativo de ZFS y DTrace.
16. **HP-UX** — *Versión: 11i v3*. UNIX de misión crítica para PA-RISC e Itanium.
17. **IBM AIX** — *Versión: 7.3*. Sistema UNIX avanzado sobre arquitectura Power Systems.
18. **SGI IRIX** — *Versión: 6.5.30*. UNIX enfocado en computación gráfica y MIPS.
19. **SCO OpenServer** — *Versión: 6.0.0*. UNIX comercial para servidores en plataformas x86.
20. **DEC Ultrix** — *Versión: 4.5*. Implementación nativa de UNIX BSD sobre VAX y DECstation.
21. **NeXTSTEP** — *Versión: 3.3*. Sistema orientado a objetos sobre Mach y BSD precursor de macOS.
22. **FreeBSD** — *Versión: 14.0-RELEASE*. Sistema BSD de alto rendimiento y servidores de red.
23. **OpenBSD** — *Versión: 7.5*. Derivado BSD enfocado en corrección, auditoría y criptografía.
24. **NetBSD** — *Versión: 10.0*. Sistema BSD con máxima portabilidad multiplataforma.
25. **DragonFly BSD** — *Versión: 6.4.0*. Variante BSD con micronúcleo híbrido y FS HAMMER2.
26. **GhostBSD** — *Versión: 24.04.1*. Sistema de escritorio intuitivo basado en FreeBSD.

Familia 3: Distribuciones GNU/Linux

27. **Debian GNU/Linux** — *Versión: 12.5 (Bookworm)*. Distribución madre comunitaria universal.
28. **Ubuntu** — *Versión: 24.04 LTS (Noble Numbat)*. Distribución con soporte extendido.
29. **Fedora Workstation** — *Versión: 40*. Innovación continua con GNOME 46 y Wayland.
30. **Red Hat Enterprise Linux** — *Versión: 9.4*. Estándar corporativo empresarial en servidores.
31. **openSUSE Leap** — *Versión: 15.6*. Distribución comunitaria alineada con SUSE Linux Enterprise.

32. **SUSE Linux Enterprise Server** — *Versión: 15 SP5*. Servidor corporativo crítico.
33. **Arch Linux** — *Versión: 2024.05.01*. Distribución minimalista bajo *rolling release*.
34. **Gentoo Linux** — *Versión: 20240505*. Sistema optimizado por compilación local con Portage.
35. **Slackware Linux** — *Versión: 15.0*. La distribución Linux activa más longeva.
36. **Alpine Linux** — *Versión: 3.20.0*. Sistema ultraligero con musl libc y BusyBox.
37. **CentOS Stream** — *Versión: 9*. Rama de integración previa para RHEL corporativo.
38. **Rocky Linux** — *Versión: 9.4*. Reconstrucción binaria comunitaria compatible con RHEL.
39. **AlmaLinux OS** — *Versión: 9.4*. Distribución empresarial gobernada sin ánimo de lucro.
40. **Linux Mint** — *Versión: 21.3 (Virginia)*. Entorno amigable con escritorio Cinnamon.
41. **Manjaro Linux** — *Versión: 24.0 (Wynsdey)*. Distribución accesible basada en Arch Linux.
42. **Void Linux** — *Versión: 20240314*. Sistema independiente con gestor XBPS e init runit.
43. **Pop!_OS** — *Versión: 22.04 LTS*. Optimizado para cómputo científico y GPU híbridas.
44. **elementary OS** — *Versión: 7.1 (Horus)*. Sistema enfocado en consistencia visual con Pantheon.
45. **Kali Linux** — *Versión: 2024.2*. Plataforma estándar en ciberseguridad y pentesting.
46. **Parrot OS** — *Versión: 6.0 (Lory)*. Entorno de seguridad informática y desarrollo forense.
47. **Puppy Linux** — *Versión: BookwormPup64 10.0.6*. Ejecución íntegra en memoria RAM.
48. **antiX Linux** — *Versión: 23.1 (Arditi del Popolo)*. Sistema ligero sin systemd con SysVinit.
49. **Lubuntu** — *Versión: 24.04 LTS*. Derivado oficial de Ubuntu con escritorio ligero LXQt.
50. **NixOS** — *Versión: 24.05 (Uakari)*. Gestión puramente funcional y declarativa.
51. **GNU Guix System** — *Versión: 1.4.0*. Distribución declarativa con Scheme y GNU Shepherd.
52. **Solus** — *Versión: 4.5 (Resilience)*. Sistema independiente para escritorio con Budgie.
53. **Mageia** — *Versión: 9*. Bifurcación comunitaria de la clásica Mandriva Linux.
54. **MX Linux** — *Versión: 23.3 (Libretto)*. Escritorio ligero y herramientas de configuración.

Familia 4: Ecosistema Microsoft y MS-DOS

55. **MS-DOS** — *Versión: 6.22*. Sistema mono-tarea en modo real de 16 bits.
56. **PC DOS** — *Versión: 7.0 Revision 1*. Edición propietaria desarrollada por IBM.
57. **FreeDOS** — *Versión: 1.3*. Reimplementación libre de código abierto compatible con DOS.
58. **Microsoft Windows 95** — *Versión: OSR 2.5 (4.00.950 C)*. Transición a 32 bits y Win32.
59. **Microsoft Windows 98** — *Versión: Second Edition (4.10)*. Soporte mejorado de USB y WDM.
60. **Microsoft Windows Me** — *Versión: 4.90.3000*. Última entrega de la rama monolítica MS-DOS.
61. **Microsoft Windows NT** — *Versión: 4.0 SP6a*. Sistema empresarial modular de 32 bits.
62. **Microsoft Windows 2000** — *Versión: Professional SP4*. Núcleo NT 5.0 y Active Directory.
63. **Microsoft Windows XP** — *Versión: Professional SP3*. Unificación de ramas sobre núcleo NT.
64. **Microsoft Windows 7** — *Versión: Ultimate SP1*. Arquitectura NT 6.1 de alta estabilidad.
65. **Microsoft Windows 10** — *Versión: Pro 22H2*. Integración de subsistema WSL y telemetría.
66. **Microsoft Windows 11** — *Versión: Pro 23H2*. Requisitos estrictos de TPM 2.0 y VBS.

Familia 5: Sistemas en tiempo real (RTOS)

67. **FreeRTOS** — *Versión: 202212.01 (LTS)*. Núcleo en tiempo real para microcontroladores.
68. **Zephyr RTOS** — *Versión: 3.6.0*. RTOS de código abierto de la Linux Foundation.
69. **Wind River VxWorks** — *Versión: 7 (SR0660)*. Sistema determinista para aviónica y sondas.
70. **QNX Neutrino RTOS** — *Versión: 7.1*. Micronúcleo crítico automotriz e industrial.
71. **RTEMS** — *Versión: 5.3*. Sistema de tiempo real para instrumentación satelital.
72. **Apache NuttX** — *Versión: 12.5.0*. RTOS compatible con estándares POSIX.
73. **eCos** — *Versión: 3.0*. Sistema embebido configurable para telecomunicaciones.
74. **RIOT OS** — *Versión: 2024.04*. Sistema de bajo consumo para nodos sensores en IoT.

75. **Contiki-NG** — *Versión: 4.9*. RTOS para redes de sensores inalámbricos 6LoWPAN.
76. **Eclipse ThreadX** — *Versión: 6.4.0*. Anteriormente Azure RTOS, certificado IEC 61508.
77. **SYSGO PikeOS** — *Versión: 5.1.3*. Hipervisor embebido y RTOS con particionado seguro.
78. **Green Hills INTEGRITY** — *Versión: 11.4.4*. RTOS certificado con nivel EAL6+ militar.
79. **LynxOS-178** — *Versión: 2.2.4*. Sistema conforme con la norma de aviónica ARINC 653.
80. **SEGGER embOS** — *Versión: V5.18.0*. Núcleo de respuesta determinista ultrarrápido.

Familia 6: Sistemas móviles y embebidos

81. **Google Android** — *Versión: 14.0 (Upside Down Cake)*. Sistema móvil sobre núcleo Linux.
82. **Apple iOS** — *Versión: 17.5.1*. Plataforma para teléfonos inteligentes sobre Darwin/XNU.
83. **Apple iPadOS** — *Versión: 17.5.1*. Variante optimizada para multitarea táctil de gran escala.
84. **Apple watchOS** — *Versión: 10.5*. Sistema operativo para dispositivos vestibles de pulsera.
85. **Apple visionOS** — *Versión: 1.2*. Sistema de cómputo espacial tridimensional en tiempo real.
86. **Huawei HarmonyOS** — *Versión: 4.2*. Plataforma distribuida multipantalla sobre microkernel.
87. **Samsung Tizen OS** — *Versión: 8.0*. Sistema para televisores inteligentes y domótica.
88. **LG webOS OSE** — *Versión: 2.25.0*. Plataforma web basada en Linux para pantallas inteligentes.
89. **Jolla Sailfish OS** — *Versión: 4.6.0 (Sauna)*. Linux europeo compatible con apps Android.
90. **KaiOS** — *Versión: 3.1*. Sistema para teléfonos básicos basado en Gecko de Mozilla.
91. **PostmarketOS** — *Versión: v23.12*. Linux para teléfonos móviles basado en Alpine Linux.
92. **Ubuntu Touch** — *Versión: OTA-4 Focal*. Sistema móvil de convergencia mantenido por UBports.
93. **BlackBerry 10 OS** — *Versión: 10.3.3.3216*. Sistema de telefonía corporativa sobre QNX.
94. **Symbian OS** — *Versión: v9.5 (Symbian^3)*. Histórico sistema para terminales Nokia.

Familia 7: Sistemas experimentales y micronúcleos

95. **Minix** — *Versión: 3.3.0*. Micronúcleo educativo multiseridor de Andrew Tanenbaum.
96. **seL4 Microkernel** — *Versión: 12.1.0*. Micronúcleo formalmente verificado por demostrador.
97. **GNU Hurd** — *Versión: 0.9*. Núcleo multiseridor oficial del proyecto GNU sobre Mach.
98. **Redox OS** — *Versión: 0.9.0*. Micronúcleo modular desarrollado íntegramente en Rust.
99. **HelenOS** — *Versión: 0.12.1*. Micronúcleo portable multiseridor académico.
100. **Haiku** — *Versión: R1/Beta 4*. Reimplementación moderna de código abierto de BeOS.
101. **SerenityOS** — *Versión: 2024-05-01*. Sistema operativo de escritorio monolítico artesanal.
102. **ReactOS** — *Versión: 0.4.14*. Reimplementación libre de la arquitectura Windows NT.
103. **KolibriOS** — *Versión: 0.7.7.0*. Sistema ultrarrápido programado en ensamblador x86.
104. **MenuetOS** — *Versión: 64-bit v1.49.80*. Sistema monousuario escrito en ensamblador x86-64.
105. **ETH Zurich Barrelfish** — *Versión: Release 2020*. Sistema multikernel para hardware disímil.
106. **Amoeba** — *Versión: 5.3*. Sistema distribuido de alta velocidad de los años noventa.
107. **CMU Mach** — *Versión: 3.0*. Micronúcleo de investigación pionero de Carnegie Mellon.
108. **TempleOS** — *Versión: 5.03*. Sistema monousuario x86-64 programado en dialecto HolyC.

Familia 8: Sistemas de red e infraestructura

109. **Cisco IOS-XE** — *Versión: 17.12.1*. Sistema de red modular de alta gama sobre Linux.
110. **Cisco NX-OS** — *Versión: 10.4(1)F*. Sistema de red para conmutadores de centros de datos.

111. **Juniper Junos OS** — *Versión: 23.4R1*. Sistema de telecomunicaciones basado en FreeBSD.
112. **MikroTik RouterOS** — *Versión: v7.14*. Plataforma para routers y conmutadores de acceso.
113. **VyOS** — *Versión: 1.4.0 (Sagitta)*. Enrutador de software modular basado en Debian.
114. **TrueNAS CORE** — *Versión: 13.0-U6*. Sistema de almacenamiento masivo ZFS sobre FreeBSD.
115. **TrueNAS SCALE** — *Versión: 23.10.2*. Almacenamiento masivo y contenedores sobre Linux.
116. **OpenWrt** — *Versión: 23.05.3*. Distribución Linux embebida para routers residenciales.
117. **Netgate pfSense Plus** — *Versión: 24.03*. Cortafuegos y enrutador de red sobre FreeBSD.
118. **OPNsense** — *Versión: 24.1*. Sistema de cortafuegos de código abierto con API REST.
119. **Proxmox VE** — *Versión: 8.2*. Virtualización basada en KVM y contenedores LXC.
120. **VMware ESXi** — *Versión: 8.0 Update 2*. Hipervisor bare-metal Tipo 1 empresarial.

3. Sistemas operativos actuales y para bajos recursos

3.1. Diez sistemas operativos actuales

En el panorama computacional contemporáneo coexisten diversas plataformas líderes que dominan los segmentos de escritorio, servidores empresariales y dispositivos móviles. Cada una de ellas implementa filosofías de diseño diferenciadas para satisfacer perfiles de usuario y exigencias de carga variables (Stallings, 2018, §2.4, p. 84).

1. Microsoft Windows 11 Pro (versión 23H2): *Características:* Núcleo híbrido NT 10.0, integración del subsistema WSL2 (Windows Subsystem for Linux), seguridad basada en virtualización (VBS) y entorno Fluent Design sobre WinUI 3. *Requerimientos mínimos:* Procesador de 64 bits con $\geqslant 2$ núcleos a 1 GHz (Intel Core 8ª gen o AMD Zen 2+), 4 GB de RAM, 64 GB de almacenamiento, firmware UEFI con Secure Boot y TPM 2.0 (Microsoft Corporation, 2024). *Perfil de usuario:* Profesionales de oficina, entornos corporativos con Active Directory, desarrolladores de la plataforma .NET/Win32 y usuarios de videojuegos de alto rendimiento (DirectX 12 Ultimate). *Ventajas y desventajas:* Ofrece una compatibilidad insuperable de software y periféricos comerciales; en contraparte, impone requisitos de hardware restrictivos que descartan equipos plenamente funcionales e integra telemetría obligatoria.

2. Apple macOS Sonoma (versión 14.5): *Características:* Núcleo híbrido XNU (microkernel Mach y subsistema FreeBSD), arquitectura optimizada para la microarquitectura ARM64 de los procesadores Apple Silicon (M1/M2/M3/M4) y emulación dinámica Rosetta 2 para binarios x86-64. *Requerimientos mínimos:* Equipo Mac propietario de Apple (desde 2018 para Intel o 2020 para Apple Silicon), 8 GB de memoria unificada y 35 GB de almacenamiento SSD disponible. *Perfil de usuario:* Desarrolladores de software para el ecosistema Apple (iOS, iPadOS, macOS), diseñadores gráficos, editores de audio y video profesional e investigadores. *Ventajas y desventajas:* Máxima eficiencia energética por vatio y estabilidad certificada UNIX (POSIX.1); en contraparte, está confinado exclusivamente al costoso ecosistema de hardware de Apple sin opción de actualización física de componentes.

3. Ubuntu 24.04 LTS (Noble Numbat): *Características:* Núcleo Linux 6.8, servidor gráfico Wayland por defecto, gestor de paquetes APT complementado con el formato de paquetes autocontenidos Snap y entorno de escritorio GNOME 46 optimizado. *Requerimientos mínimos:* Procesador de 64 bits de doble núcleo a 2 GHz, 4 GB de memoria RAM y 25 GB de almacenamiento en disco. *Perfil de usuario:* Desarrolladores de aplicaciones en la

nube, científicos de datos, ingenieros de devops y usuarios generales que buscan una plataforma Linux amigable y robusta. *Ventajas y desventajas:* Inmenso repositorio de software, enorme comunidad de soporte y ciclo predecible de cinco años de soporte técnico oficial; su principal desventaja radica en la penalización de rendimiento y sobrecarga inicial de las aplicaciones distribuidas como paquetes Snap.

4. Debian GNU/Linux 12.5 (Bookworm): *Características:* Núcleo Linux 6.1 LTS, sistema de empaquetado DPKG/APT, sistema de inicio systemd, política rigurosa de software libre (DFSG) y soporte nativo para más de nueve arquitecturas de hardware (The Debian Project, 2023). *Requerimientos mínimos:* Procesador de 1 GHz, 1 GB de RAM (512 MB en modo servidor consola; 2 GB para entorno gráfico estándar) y 10 GB de disco. *Perfil de usuario:* Administradores de servidores de producción crítica, ingenieros de infraestructura y usuarios que priorizan la estabilidad incondicional y la independencia de corporaciones. *Ventajas y desventajas:* Estabilidad de roca probada en producción y consumo eficiente de memoria; como desventaja, los paquetes de software y controladores en sus repositorios estables sacrifican la novedad frente a la estabilidad.

5. Fedora Workstation 40: *Características:* Núcleo Linux 6.8, entorno de escritorio GNOME 46 puro, gestor de paquetes DNF5 sobre formato RPM, sistema de archivos Btrfs con compresión zstd transparente y servidor de sonido PipeWire. *Requerimientos mínimos:* Procesador de 64 bits a 2 GHz multinúcleo, 4 GB de RAM y 20 GB de espacio en almacenamiento secundario. *Perfil de usuario:* Ingenieros de software, desarrolladores de código abierto y profesionales que requieren acceso inmediato a los últimos estándares tecnológicos del ecosistema Linux. *Ventajas y desventajas:* Innovación continua y adopción temprana de estándares de vanguardia (Wayland estricto, PipeWire, Btrfs); su desventaja radica en un ciclo de vida corto por versión (13 meses), que exige migraciones frecuentes.

6. Red Hat Enterprise Linux 9.4 (RHEL): *Características:* Núcleo Linux 5.14 con portabilidad retroactiva de parches de seguridad (*backporting*), módulos de seguridad SELinux en modo obligatorio (*enforcing*) y gestor de contenedores Podman integrado sin demonio central. *Requerimientos mínimos:* Procesador compatible con la especificación de microarquitectura x86-64-v2, 2 GB de RAM (4 GB recomendados) y 20 GB de almacenamiento. *Perfil de usuario:* Grandes corporaciones, banca financiera, centros de cómputo de alta criticidad e infraestructuras gubernamentales. *Ventajas y desventajas:* Soporte comercial corporativo de hasta diez años y certificaciones de cumplimiento de seguridad de grado militar (FIPS 140-3, Common Criteria); su principal desventaja es el costo elevado de licenciamiento para implementaciones productivas.

7. Arch Linux (versión continua): *Características:* Modelo de liberación continua (*rolling release*), núcleo Linux estándar de compilación rápida, gestor de paquetes pacman, repositorios de usuarios AUR y principio de diseño KISS (*Keep It Simple, Stupid*). *Requerimientos mínimos:* Procesador x86-64, 512 MB de RAM (para consola de instalación; $\geqslant 2$ GB para entorno gráfico) y 2 GB de disco para el sistema base. *Perfil de usuario:* Usuarios avanzados de Linux, programadores e ingenieros que desean ensamblar un sistema operativo estrictamente personalizado desde la línea de comandos. *Ventajas y desventajas:* Control total sobre el software instalado, sin bloatware ni componentes innecesarios, junto con una documentación enciclopédica mundial (ArchWiki); exige conocimientos avanzados de administración y riesgo de regresiones si no se gestionan las actualizaciones con cuidado.

8. FreeBSD 14.0-RELEASE: *Características:* Sistema operativo completo derivado de UNIX BSD con núcleo monolítico modular y espacio de usuario unificado, sistema de archivos ZFS integrado de fábrica, virtualización bhyve y aislamiento mediante contenedores Jails (The

FreeBSD Project, 2023). *Requerimientos mínimos:* Procesador x86-64 o AArch64, 1 GB de RAM (mínimo 4 GB si se utiliza ZFS con deduplicación) y 4 GB de disco. *Perfil de usuario:* Administradores de servidores de almacenamiento masivo (NAS), ingenieros de telecomunicaciones, proveedores de red (ISP) y entornos que requieren procesamiento de paquetes de red de alto caudal. *Ventajas y desventajas:* Extraordinaria pila de red TCP/IP y confiabilidad superior de ZFS para integridad de datos; presenta menor soporte de controladores para tarjetas inalámbricas y chips gráficos de última hornada orientados al consumidor general.

9. Google Android 14 (Upside Down Cake): *Características:* Núcleo Linux alineado con la especificación GKI (Generic Kernel Image), entorno de ejecución ART (Android Runtime) basado en compilación anticipada (AOT) y Just-in-Time (JIT), y comunicación interprocesos mediante el subsistema Binder. *Requerimientos mínimos:* SoC ARM64 o x86-64 con arquitectura de 64 bits, 4 GB de memoria RAM (2 GB en la variante optimizada Android Go Edition) y 32 GB de memoria flash. *Perfil de usuario:* Usuarios globales de telefonía móvil inteligente, tabletas táctiles, dispositivos vestibles y sistemas de información en vehículos. *Ventajas y desventajas:* La mayor base instalada de aplicaciones del planeta y ecosistema de conectividad universal; sufre de fragmentación grave en los ciclos de actualización dependientes de los fabricantes de hardware.

10. Google ChromeOS (versión 124+): *Características:* Núcleo Linux con interfaz basada en el navegador Chromium, aislamiento estricto de aplicaciones en cajones de arena (*sandboxing*), compatibilidad con aplicaciones de Android mediante contenedor ARCVM y subsistema Linux Crostini. *Requerimientos mínimos:* Procesador x86-64 o ARM de bajo consumo, 4 GB de RAM y 32 GB de almacenamiento flash eMMC o SSD. *Perfil de usuario:* Estudiantes, entornos escolares, personal administrativo de oficina y usuarios cuyo flujo de trabajo es enteramente dependiente de servicios web en la nube. *Ventajas y desventajas:* Arranque ultraveloz en pocos segundos, sistema de archivos de solo lectura inmune a malware común y mantenimiento cero; desventaja crítica por su alta dependencia de la conectividad a internet e incapacidad para ejecutar software de ingeniería pesado fuera de la nube.

3.2. Diez sistemas operativos para bajos recursos

La obsolescencia programada y los crecientes requisitos de hardware de los sistemas comerciales han impulsado el desarrollo de distribuciones especializadas en revitalizar equipos antiguos o con recursos de memoria y procesamiento severamente restringidos.

1. Puppy Linux (BookwormPup64 10.0.6): *Características:* Sistema diseñado para cargarse enteramente en la memoria RAM durante el proceso de arranque, utilizando el gestor de ventanas ligero JWM y el administrador de archivos ROX-Filer. *Requerimientos mínimos:* Procesador x86-64 o x86 a 1 GHz, 512 MB de RAM (1 GB recomendado para navegación web) y 1 GB en disco o memoria USB. *Perfil de usuario:* Técnicos de mantenimiento, usuarios que desean rescatar ordenadores portátiles antiguos y entornos donde se requiere un sistema portable de rescate sin desgaste del disco. *Ventajas y desventajas:* Velocidad de respuesta instantánea al operar al 100 % desde la RAM física; la interfaz gráfica resulta arcaica y la configuración de persistencia requiere almacenamiento en archivos especiales de sesión.

2. antiX Linux 23.1 (Arditi del Popolo): *Características:* Distribución basada en Debian Stable completamente libre del sistema de inicialización systemd (emplea SysVinit o runit

para maximizar el ahorro de memoria), combinada con entornos de ventanas minimalistas como IceWM, Fluxbox o JWM. *Requerimientos mínimos:* Procesador Intel Pentium III o superior (soporta x86 de 32 bits y 64 bits), 256 MB de RAM (512 MB recomendados) y 5 GB de espacio en disco. *Perfil de usuario:* Propietarios de computadoras de más de quince años de antigüedad y usuarios que exigen mínima sobrecarga de servicios en segundo plano. *Ventajas y desventajas:* Huella de memoria en reposo sorprendentemente reducida (inferior a 160 MB de RAM); carece de la automatización de ciertos servicios modernos que dependen intrínsecamente de systemd.

3. Lubuntu 24.04 LTS: *Características:* Variante oficial de Ubuntu construida sobre el entorno de escritorio LXQt (bibliotecas Qt optimizadas) y el gestor de ventanas Openbox, sustituyendo los componentes pesados de GNOME por aplicaciones ligeras. *Requerimientos mínimos:* Procesador de 64 bits a 1.5 GHz, 1 GB de memoria RAM (2 GB para navegación web moderna) y 10 GB de disco. *Perfil de usuario:* Usuarios domésticos y de oficina que buscan el soporte y los repositorios de Ubuntu en equipos con memoria y procesamiento moderados. *Ventajas y desventajas:* Acceso al catálogo masivo de software de Ubuntu con un consumo de recursos muy inferior; no obstante, requiere procesadores de 64 bits y un mínimo de 1 GB de RAM, dejando fuera hardware extremadamente obsoleto.

4. Alpine Linux 3.20.0: *Características:* Distribución no GNU construida sobre la biblioteca de lenguaje C reducida musl libc y las utilerías unificadas de BusyBox, orientada a la seguridad con compilación de pila protegida por defecto. *Requerimientos mínimos:* Procesador x86, x86-64 o ARM, 128 MB de RAM (64 MB en modo servidor de consola) y 500 MB de disco. *Perfil de usuario:* Desarrolladores de contenedores ligeros (imágenes base de Docker), ingenieros de sistemas embebidos y administradores de enrutadores o cortafuegos de bajo consumo. *Ventajas y desventajas:* Consumo casi imperceptible de recursos y tamaño de imagen base de apenas 5 MB; su desventaja es la incompatibilidad binaria con programas precompilados que dependen de las funciones propietarias de glibc.

5. Tiny Core Linux 15.0: *Características:* Plataforma minimalista extrema cuyo núcleo Linux y utilerías BusyBox ocupan entre 16 y 22 MB con servidor gráfico FLTK/FLWM. El sistema se ejecuta al 100 % en RAM y carga extensiones modulares bajo demanda sin escribir en disco. *Requerimientos mínimos:* Procesador Intel 486DX o superior, 46 MB de memoria RAM (128 MB en modo gráfico) y 50 MB de almacenamiento. *Perfil de usuario:* Investigadores, ingenieros de sistemas empotrados y entusiastas de la eficiencia computacional pura. *Ventajas y desventajas:* El sistema gráfico más pequeño del mundo con soporte vigente; requiere una curva de aprendizaje pronunciada para configurar la persistencia de datos y extensiones de software.

6. Bodhi Linux 7.0.0: *Características:* Basada en Ubuntu LTS, implementa el entorno de escritorio Moksha (bifurcación depurada y optimizada del entorno Enlightenment 17), que proporciona composiciones visuales atractivas sin sobrecargar la CPU. *Requerimientos mínimos:* Procesador de 64 bits a 1 GHz (conserva una edición Legacy de 32 bits sin soporte PAE), 512 MB de RAM y 5 GB de almacenamiento. *Perfil de usuario:* Usuarios que buscan una apariencia estética moderna y efectos visuales fluidos en hardware antiguo que no tolera entornos pesados como GNOME o Windows 10. *Ventajas y desventajas:* Interfaz visual elegante con un consumo en reposo cercano a los 220 MB de RAM; incluye muy poco software de fábrica, requiriendo configuración inicial por parte del usuario.

7. Q4OS 5.4 (Aquarius): *Características:* Distribución sustentada en Debian Stable que integra el entorno Trinity Desktop Environment (TDE), una bifurcación optimizada del código clásico de KDE 3.5 con aspecto y respuesta similares a los sistemas de principios de siglo.

Requerimientos mínimos: Procesador Intel Pentium a 300 MHz o procesador x86-64, 256 MB de RAM (edición Trinity) o 1 GB (edición Plasma), y 3 GB de disco. *Perfil de usuario:* Usuarios habituados a la ergonomía de interfaces clásicas de escritorio que desean revivir equipos antiguos con una base de software moderna y mantenida. *Ventajas y desventajas:* Excepcional estabilidad heredada de Debian y familiaridad inmediata para el usuario de interfaces clásicas; el catálogo de aplicaciones nativas de Trinity es más reducido que el de GTK.

8. Slax 15.0.4: *Características:* Sistema operativo modular portable basado en Debian que utiliza el gestor de ventanas Fluxbox y un sistema de archivos comprimido por capas SquashFS con soporte de lectura/escritura mediante OverlayFS. *Requerimientos mínimos:* Procesador x86 o x86-64 a 1 GHz, 256 MB de RAM (512 MB para navegar en la web) y 1 GB en medio extraíble USB o disco. *Perfil de usuario:* Técnicos de soporte, usuarios móviles y operadores de campo que requieren un escritorio completo que quepa en un pendrive de 1 GB. *Ventajas y desventajas:* Arquitectura modular donde cualquier aplicación adicional se agrega como un único archivo comprimido sin instalación; no es óptimo para configuraciones complejas de estación fija permanente.

9. BunsenLabs Linux (Boron): *Características:* Continuadora directa de la distribución CrunchBang Linux, estructurada sobre Debian Stable con una configuración optimizada del gestor de ventanas Openbox, el panel tint2 y el monitor de sistema Conky. *Requerimientos mínimos:* Procesador x86-64 a 1 GHz (mantiene instalador de 32 bits), 512 MB de RAM (1 GB recomendado) y 10 GB de disco. *Perfil de usuario:* Desarrolladores, administradores de sistemas y entusiastas del teclado que buscan una experiencia de uso ágil, fluida y orientada a atajos. *Ventajas y desventajas:* Configuración de fábrica refinada con un consumo en reposo inferior a 190 MB de RAM; requiere que el usuario esté cómodo editando archivos de texto para personalizar atajos y menús.

10. Peppermint OS (Bookworm): *Características:* Sistema modular basado en Debian Stable que utiliza una combinación ligera de XFCE y Openbox, integrando la utilidad Ice para encapsular sitios y herramientas web como aplicaciones de escritorio nativas (SSB, *Site-Specific Browser*). *Requerimientos mínimos:* Procesador de 64 bits a 1 GHz, 1 GB de RAM (2 GB para navegación fluida) y 10 GB de almacenamiento. *Perfil de usuario:* Usuarios que prefieren construir su sistema a medida a partir de un entorno limpio sin programas preinstalados innecesarios, integrando servicios en la nube en su escritorio. *Ventajas y desventajas:* Ausencia total de bloatware y control absoluto del usuario sobre qué instalar; exige trabajo manual de configuración inicial para usuarios novatos.

4. Virtualización de sistemas operativos

4.1. Conceptos e hipervisores

La virtualización es una técnica de abstracción computacional mediante la cual los recursos físicos de una máquina (procesadores, memoria principal, dispositivos de almacenamiento y adaptadores de red) se desacoplan de los entornos de ejecución que operan sobre ellos (Stallings, 2018, §14.1, p. 628). Mediante este desacoplamiento, una única estación física o servidor puede albergar de manera concurrente múltiples instancias aisladas de sistemas operativos completos, denominadas máquinas virtuales (VM), cada una de las cuales percibe la ilusión de poseer un conjunto exclusivo e ininterrumpido de componentes de hardware (Silberschatz et al., 2006, §2.8, pp. 58–59).

El componente central que viabiliza esta separación es el monitor de máquina virtual (VMM, *Virtual Machine Monitor*) o hipervisor. Esta capa de software o microcódigo actúa como árbitro y distribuidor de recursos físicos entre los sistemas huéspedes (*guest OS*), interceptando y gestionando las instrucciones privilegiadas de la CPU que, en condiciones nativas, accederían de forma directa a los registros del procesador y a las tablas de páginas de memoria (Stallings, 2018, §14.2, p. 631). Atendiendo a la relación que guarda el hipervisor con el hardware anfitrión, la arquitectura computacional distingue dos categorías fundamentales:

1. **Hipervisores Tipo 1 (nativos o *bare-metal*):** Se ejecutan directamente sobre los circuitos electrónicos del sistema físico, sin la intermediación de un sistema operativo anfitrión previo. El hipervisor asume el control absoluto de la CPU, la gestión de la memoria física y las interrupciones del hardware. Debido a que eliminan capas intermedias de traducción, ofrecen latencias mínimas y un rendimiento cercano al de la ejecución sobre hardware nativo, convirtiéndose en el estándar dominante en la infraestructura de computación en la nube y los centros de datos corporativos (Stallings, 2018, §14.2, p. 632).

2. **Hipervisores Tipo 2 (alojados o *hosted*):** Se implementan como aplicaciones ordinarias de espacio de usuario sobre un sistema operativo anfitrión preexistente (*host OS*), como Linux, Windows o macOS. Las peticiones de cómputo, memoria y E/S generadas por el sistema huésped deben ser canalizadas y convertidas a través del subsistema de llamadas al sistema del anfitrión. Si bien esta estructura añade una sobrecarga por conmutación de contexto adicional, proporciona una enorme flexibilidad para labores de desarrollo, emulación, análisis de seguridad y pruebas de software en estaciones de trabajo de escritorio (Stallings, 2018, §14.2, p. 633).

Para maximizar la eficiencia en la ejecución de instrucciones del procesador, los hipervisores modernos aprovechan extensiones de silicio (tales como Intel VT-x y AMD-V), las cuales introducen modos operativos adicionales que permiten la virtualización completa sin requerir modificaciones en el código fuente del sistema operativo huésped, reservando técnicas complementarias como la paravirtualización para la optimización de controladores de red y almacenamiento en bloque mediante hiperllamadas (*hypercalls*) directas.

4.2. Software de virtualización

La implementación práctica de entornos virtualizados abarca un amplio espectro de soluciones de software, clasificadas rigurosamente según el nivel de abstracción que introducen y el entorno operativo para el que fueron concebidas. En el ámbito corporativo y de centros de cómputo de alta densidad predominan los hipervisores Tipo 1 de microcódigo o núcleo dedicado, como VMware ESXi, la plataforma de código abierto Proxmox VE y Microsoft Hyper-V, diseñados para soportar migración en vivo de máquinas virtuales, balanceo dinámico de cargas y redundancia por clústeres. En contraste, en el desarrollo local y la investigación académica prevalecen hipervisores Tipo 2 y entornos de emulación dinámica, destacando Oracle VM VirtualBox, VMware Workstation Pro y QEMU, que permiten instanciar sistemas operativos disímiles con mínima fricción operativa. Asimismo, este ecosistema se extiende hacia las tecnologías de aislamiento a nivel de núcleo, donde los contenedores (Docker Engine, LXC) comparten un mismo kernel para eliminar el costo de instanciar un sistema operativo completo.

La estructuración de las principales herramientas y tecnologías de software que permiten virtualizar sistemas operativos se detalla de forma sistemática en el cuadro sinóptico de la Imagen 2.

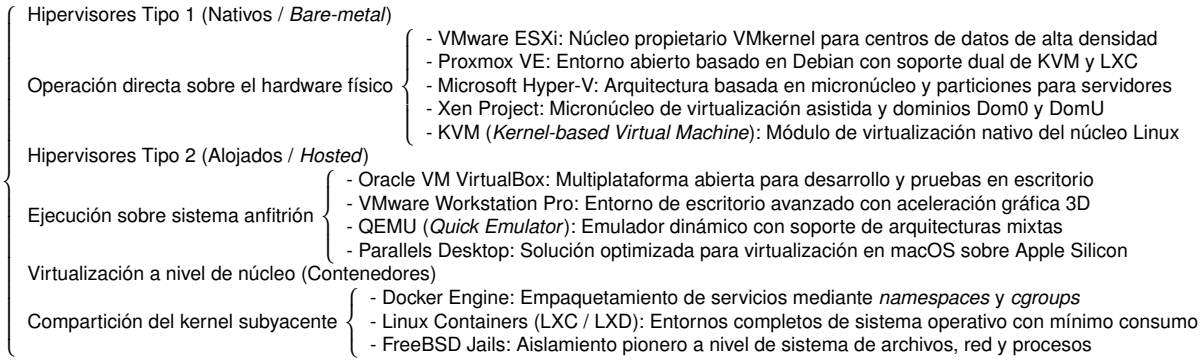


Imagen 2: Clasificación del software de virtualización y aislamiento de sistemas operativos.

5. Procesos en C:\Windows\System32

5.1. Arquitectura y subsistemas de ejecución en System32

El directorio `C:\Windows\System32` constituye el núcleo operativo fundamental en la arquitectura de la familia de sistemas operativos Windows NT, agrupando las bibliotecas de enlace dinámico (*Dynamic Link Libraries*, DLL), los controladores esenciales del núcleo, los servicios del sistema y las herramientas de administración en línea de comandos (Deitel et al., 2004, §21.2, pp. 1038–1039). En las implementaciones modernas de 64 bits (arquitecturas x86-64 y ARM64), este directorio alberga los ejecutables nativos compilados para el espacio de direcciones de 64 bits, mientras que las versiones de 32 bits de dichas herramientas se preservan de manera paralela en el directorio `SysWOW64` bajo la mediación del subsistema de emulación WoW64 (*Windows-on-Windows 64-bit*).

La arquitectura de ejecución divide estrictamente las responsabilidades entre el espacio de núcleo (*kernel mode*) y el espacio de usuario (*user mode*) (Silberschatz et al., 2006, §22.2, pp. 783–785). Los programas que residen directamente en `System32` abarcan desde procesos del sistema de máxima confianza que interactúan con el núcleo ejecutivo (`ntoskrnl.exe`) y la capa de abstracción de hardware (HAL), hasta utilidades de línea de comandos, intérpretes de órdenes y consolas administrativas estructuradas en el formato estándar Portable Executable (PE32/PE32+). La coordinación de las herramientas de consola involucra al subsistema de tiempo de ejecución cliente/servidor (`csrss.exe`) y al proceso anfitrión de consola (`conhost.exe`), el cual administra los búferes de pantalla, la codificación tipográfica y los flujos estándar de entrada, salida y error (`STDIN`, `STDOUT` y `STDERR`).

Asimismo, la resolución de ejecutables dentro del intérprete de órdenes (`cmd.exe`) está regida por la variable de entorno `%PATHEXT%`. Dado que su configuración predeterminada asigna prioridad a las extensiones `.COM` por encima de `.EXE` (siguiendo el orden `.COM`, `.EXE`, `.BAT`, `.CMD`), el sistema garantiza la compatibilidad hacia atrás con guiones por lotes históricos, asegurando que las herramientas fundamentales del sistema operativo se ejecuten de forma prioritaria y evitando colisiones de nombres.

5.2. Procesos críticos del sistema y del núcleo

En el espacio de usuario de Windows NT operan diversos procesos residentes permanentes en `System32`, indispensables para la inicialización del sistema, la gestión de sesiones, la seguridad criptográfica y el soporte de la interfaz gráfica (Stallings, 2018, §2.5, pp. 88–93):

1. **smss.exe (*Session Manager Subsystem*):** Es el primer proceso en modo de usuario generado durante la secuencia de arranque, invocado directamente por el hilo inicial del núcleo (Silberschatz et al., 2006, §22.3, p. 786). Sus funciones medulares comprenden la definición de variables globales de entorno, la inicialización de los archivos de paginación de memoria virtual (`pagefile.sys`), la carga de los subsistemas requeridos y la creación de las sesiones independientes de usuario (sesión 0 para servicios y sesión 1 para el usuario interactivo), instanciando a su vez los procesos `csrss.exe` y `wininit.exe`.

2. **csrss.exe (*Client/Server Runtime Subsystem*):** Proceso esencial responsable de mantener el subsistema de entorno Win32/Win64. Administra la creación, ciclo de vida y destrucción de procesos e hilos en espacio de usuario, supervisa las políticas de terminación de aplicaciones y suministra soporte crítico al subsistema gráfico y de consolas (Deitel et al., 2004, §21.3, p. 1042).

3. **wininit.exe (*Windows Initialization Process*):** Proceso de inicialización confinado a la sesión aislada 0. Su función primordial radica en arrancar y supervisar los pilares del espacio de servicios: el Administrador de Control de Servicios (`services.exe`), el subsistema de seguridad local (`lsass.exe`) y el Administrador de Sesiones Locales (`lsm.exe`). Su terminación anómala provoca de inmediato una verificación de error del núcleo (*bug check* o pantalla azul).

4. **services.exe (*Service Control Manager*, SCM):** Actúa como el motor administrativo central de todos los servicios del sistema y controladores de dispositivo que operan en segundo plano de manera no interactiva. Mantiene la base de datos en memoria del estado de los servicios, gestiona sus dependencias jerárquicas y ejecuta las políticas de recuperación automática y reinicio ante fallos.

5. **lsass.exe (*Local Security Authority Subsystem Service*):** Proceso medular de la arquitectura de seguridad de Windows NT (Deitel et al., 2004, §21.7, p. 1070). Se encarga de hacer cumplir las políticas de seguridad locales, verificar la autenticación de usuarios mediante paquetes criptográficos (Kerberos y NTLM), generar los identificadores de seguridad (SID) y tokens de acceso para los nuevos procesos, y registrar los eventos en el registro de auditoría de seguridad.

6. **winlogon.exe (*Windows Logon Process*):** Administrador de las sesiones interactivas de usuario. Controla los estados de autenticación, bloqueo, desbloqueo y cambio de usuario, captura la secuencia de atención segura (SAS, *Secure Attention Sequence*, tradicionalmente Ctrl+Alt+Del) para impedir suplantaciones por software malicioso, e inicializa el entorno de escritorio del usuario mediante `userinit.exe`.

7. **svchost.exe (*Service Host Process*):** Proceso contenedor genérico diseñado para ejecutar múltiples servicios del sistema implementados en bibliotecas DLL dinámicas. Agrupa los servicios en diferentes instancias de proceso clasificadas por niveles de privilegio (LocalSystem, NetworkService, LocalService) para optimizar el consumo de memoria y aislar posibles fallos de ejecución.

8. **dwm.exe (*Desktop Window Manager*):** Gestor de composición de escritorio asistido por aceleración gráfica por hardware. Renderiza cada ventana en búferes de memoria de vídeo independientes fuera de pantalla mediante interfaces Direct3D y DirectComposition antes de componer la escena visual final, posibilitando efectos de translucidez, animaciones fluidas y eliminación de artefactos gráficos de repintado.

9. **conhost.exe (*Console Window Host*):** Proceso mediador que aloja la interfaz gráfica, búferes de entrada/salida y renderizado de texto para programas de consola, desvinculando esta tarea de `csrss.exe` para mitigar vectores de explotación y garantizar el aislamiento entre clientes de consola.

10. **fontdrvhost.exe (*Usermode Font Driver Host*):** Proceso de seguridad que aísla los controladores de renderizado de fuentes tipográficas en espacio de usuario, impidiendo que vulnerabilidades en el procesamiento de formatos tipográficos complejos comprometan el núcleo del sistema.

11. **sihost.exe (*Shell Infrastructure Host*):** Proceso de soporte que asiste en la composición y comportamiento de los elementos estructurales de la interfaz de usuario moderna, tales como el menú de inicio, la barra de tareas y el centro de notificaciones.

12. **taskhostw.exe (*Host Process for Windows Tasks*):** Contenedor anfitrión genérico para la ejecución programada de tareas y eventos del sistema operativo sustentados en bibliotecas dinámicas.

13. **RuntimeBroker.exe:** Administrador de intermediación de permisos y capacidades de seguridad, encargado de validar y auditar el acceso de las aplicaciones de la plataforma universal (UWP y WinUI) a periféricos y sensores (micrófono, cámara, ubicación geográfica).

14. **explorer.exe:** Shell principal del entorno interactivo de Windows. Aunque su ejecutable maestro reside en `C:\Windows`, sus módulos y procesos auxiliares vinculados a la administración de ventanas, bandeja del sistema y exploración jerárquica de archivos operan en estrecha coordinación con las librerías de `System32`.

5.3. Utilerías ejecutables .com

Dentro del catálogo de `System32` subsisten herramientas con la extensión histórica `.com`, las cuales, a diferencia de los binarios planos de 16 bits de MS-DOS (limitados a un único segmento de 64 KB a partir del PSP en `0x0100`), son ejecutables nativos en formato Portable Executable enlazados al subsistema CUI (Deitel et al., 2004, §21.2, p. 1037):

1. **chcp.com (*Change Code Page*):** Proceso responsable de consultar y reconfigurar la tabla o página de códigos activa para el búfer de entrada y salida de la sesión de consola (Microsoft Corporation, 2023). Invoca de forma programática las interfaces Win32 `GetConsoleCP`, `GetConsoleOutputCP`, `SetConsoleCP` y `SetConsoleOutputCP`. Entre las tablas representativas destacan la Página 437 (OEM Estados Unidos con caracteres semigráficos), Página 850 (OEM Multilingüe Latino 1), Página 1252 (Windows ANSI) y Página 65001 (UTF-8 universal). Su sintaxis abarca la consulta directa (`chcp`) o la reasignación numérica (`chcp [código]`).

2. **format.com (*Format Disk Volume*):** Utilidad de bajo nivel orientada a estructurar lógicamente un volumen o partición de almacenamiento en bloque, preparando los metadatos de un sistema de archivos específico (Microsoft Corporation, 2023). Interactúa con el servicio de discos virtuales (VDS) y los controladores del núcleo (`ntfs.sys`, `fastfat.sys`, `exfat.sys`, `refs.sys`). Bloquea de forma exclusiva el volumen mediante `FSCTL_LOCK_VOLUME`, fuerza su desmontaje con `FSCTL_DISMOUNT_VOLUME`, inicializa el sector de arranque (VBR) y genera las tablas de asignación (MFT en NTFS o FAT1/FAT2). Su sintaxis estándar responde a `format volumen [/FS:sistema] [/V:etiqueta] [/Q] [/A:tamaño]`.

3. **mode.com (*Configure System Devices*):** Herramienta de parametrización de dispositivos físicos y puertos de comunicación, interactuando con las capas de abstracción de controladores mediante la manipulación del bloque de control de dispositivo (DCB) vía `SetCommState` y `SetConsoleScreenBufferSize` (Microsoft Corporation, 2023). Permite regular parámetros de líneas serie COM (`mode COMm[:] [BAUD=b] [PARITY=p] [DATA=d] [STOP=s]`), geometría de la pantalla de texto (`mode CON[:] [COLS=c] [LINES=n]`), selección de páginas tipográficas en impresoras locales y la tasa de repetición del teclado (*typematic rate*).

4. **tree.com (*Directory Tree Viewer*):** Proceso utilitario que genera una proyección visual jerárquica y recursiva en forma de árbol de los directorios y subdirectorios de una unidad de almacenamiento, con soporte para desplegar nombres de archivo mediante el modificador `/F` y codificación ASCII con `/A`.

5. **more.com (*Output Pager*):** Filtro y paginador de texto estándar que segmenta flujos masivos de caracteres procedentes de la entrada estándar o de archivos en pantallas sucesivas, pausando la ejecución hasta la pulsación de una tecla para permitir la lectura pausada.

6. **diskcomp.com (*Disk Compare*):** Utilidad heredada diseñada para efectuar la comparación física binaria sector por sector entre dos medios de almacenamiento extraíbles compatibles, verificando su total concordancia.

7. **diskcopy.com (*Disk Copy*):** Herramienta de duplicación de almacenamiento magnético que copia de manera exhaustiva el contenido de un medio de origen hacia uno de destino a nivel de sectores físicos absolutos.

5.4. Catálogo de programas y herramientas de System32

El repertorio de ejecutables residentes directamente en `C:\Windows\System32` abarca cientos de utilidades especializadas que sustentan la administración, el diagnóstico y la operación del sistema operativo. A continuación se presentan estructuradas de forma sistemática por áreas funcionales (Microsoft Corporation, 2023):

5.4.1. Almacenamiento, sistemas de archivos y particiones

1. **chkdsk.exe (*Check Disk*):** Inspecciona la integridad estructural y geométrica de sistemas de archivos FAT y NTFS, verificando descriptores de seguridad, índices de directorios y reparando sectores dañados mediante reasignación de clústeres.

2. **diskpart.exe (*Disk Partition Tool*):** Intérprete interactivo para la administración avanzada de particiones de disco (MBR y GPT), volúmenes dinámicos, creación y acoplamiento de discos virtuales VHD/VHDX y asignación de atributos de almacenamiento.
3. **fsutil.exe (*File System Utility*):** Potente herramienta técnica para consultar y manipular propiedades del sistema de archivos, gestionar puntos de reanálisis (*reparse points*), controlar archivos dispersos (*sparse files*) y configurar cuotas de disco.
4. **defrag.exe (*Disk Defragmenter*):** Optimiza el rendimiento de lectura y escritura reorganizando bloques contiguos en discos magnéticos y ejecutando comandos TRIM en unidades de estado sólido (SSD).
5. **compact.exe (*File Compression Utility*):** Administra de forma granular la compresión de archivos y carpetas nativa sobre particiones con formato NTFS.
6. **cipher.exe (*Encrypting File System Utility*):** Administra el cifrado de datos mediante EFS (*Encrypting File System*), gestiona certificados criptográficos y ejecuta sobreescritura segura de espacio libre para impedir recuperación forense.
7. **vssadmin.exe (*Volume Shadow Copy Administrative Tool*):** Administra proveedores de instantáneas de volumen (*shadow copies*) y copias de seguridad de solo lectura del estado del sistema.
8. **fltmc.exe (*Filter Manager Control*):** Administra y depura controladores de filtro del sistema de archivos (*minifilters*), esenciales para motores antivirus, auditoría y cifrado en tiempo real.
9. **mountvol.exe (*Mount Volume Utility*):** Crea, elimina y lista puntos de montaje de volúmenes asociados a identificadores únicos GUID sin requerir una letra de unidad.
10. **robocopy.exe (*Robust File Copy*):** Motor avanzado de réplica y sincronización de directorios tolerante a fallos de red, con soporte para reanudación, multihilo y conservación de metadatos NTFS.
11. **xcopy.exe (*Extended File Copy*):** Copia recursiva de archivos y árboles de directorios completos preservando atributos de archivo y permisos.
12. **attrib.exe (*File Attribute Utility*):** Modifica y consulta los atributos fundamentales de archivos y directorios: solo lectura (R), archivo (A), sistema (S) y oculto (H).
13. **label.exe (*Disk Label Utility*):** Crea, modifica o elimina la etiqueta de identificación textual asignada a un volumen lógico.
14. **subst.exe (*Substitute Path Utility*):** Asocia una ruta de directorio local o de red con una letra de unidad de disco virtual.
15. **cleanmgr.exe (*Disk Space Cleanup Manager*):** Herramienta automatizada de saneamiento y liberación de espacio en disco que analiza y purga archivos temporales, registros y descargas antiguas.

5.4.2. Redes, comunicaciones y diagnóstico

1. **ipconfig.exe (*IP Configuration*):** Reporta la configuración de los adaptadores de red (IPv4, IPv6, máscara de subred, puerta de enlace) y gestiona la liberación y renovación de concesiones DHCP y vaciado de caché DNS (`/flushdns`).
2. **ping.exe (*Packet Internet Groper*):** Diagnostica la accesibilidad de nodos de red enviando paquetes de solicitud de eco ICMP (*Echo Request*) y midiendo el tiempo de ida y vuelta (*RTT*).
3. **tracert.exe (*Trace Route*):** Determina la ruta física y lógica de encaminamiento hacia un destino remoto manipulando el campo tiempo de vida (TTL) de los paquetes IP.
4. **pathping.exe (*Path Ping*):** Combina la funcionalidad de trazado de ruta con estadísticas

prolongadas de latencia y pérdida de paquetes en cada salto intermedio.

5. **netstat.exe (*Network Statistics*):** Muestra conexiones TCP activas, puertos en escucha (listening), tablas de enrutamiento IP y estadísticas del tráfico de red.
6. **route.exe (*IP Routing Table Management*):** Permite visualizar, añadir, modificar y eliminar entradas estáticas en la tabla de enrutamiento IP del sistema local.
7. **arp.exe (*Address Resolution Protocol Tool*):** Consulta y manipula las tablas de correspondencia entre direcciones de capa de red (IP) y direcciones físicas (MAC).
8. **nslookup.exe (*Name Server Lookup*):** Consulta interactiva a servidores DNS para resolver nombres de host, direcciones IP y registros de recursos (A, AAAA, MX, PTR).
9. **netsh.exe (*Network Shell*):** Entorno de línea de comandos exhaustivo para configurar interfaces de red, cortafuegos de Windows, redes inalámbricas WLAN y parámetros TCP/IP.
10. **nbtstat.exe (*NetBIOS Statistics*):** Diagnostica la resolución de nombres NetBIOS sobre TCP/IP y muestra tablas de nombres locales y remotos.
11. **net.exe (*Network Command Utility*):** Utilidad multipropósito para administrar recursos compartidos de red (`net share`), cuentas de usuario (`net user`), grupos locales y control de servicios.
12. **getmac.exe (*Get MAC Address*):** Detecta y presenta las direcciones MAC de todos los adaptadores de red instalados en el equipo local o remoto.
13. **hostname.exe (*Host Name Query*):** Devuelve de manera directa el nombre NetBIOS y DNS asignado a la computadora local.

5.4.3. Gestión de procesos, rendimiento y supervisión

1. **tasklist.exe (*Task List*):** Proyecta un listado completo de todos los procesos en ejecución, indicando su identificador PID, nombre de imagen binaria, sesión y memoria ocupada.
2. **taskkill.exe (*Task Kill*):** Permite finalizar procesos de forma grácil o forzada especificando su identificador numérico (PID) o nombre de imagen binaria.
3. **taskmgr.exe (*Task Manager*):** Monitor visual completo de recursos del sistema, supervisión de procesos, hilos, rendimiento de hardware, historial de aplicaciones y programas de inicio.
4. **perfmon.exe (*Performance Monitor*):** Consola avanzada de supervisión de contadores de rendimiento en tiempo real y generación de registros analíticos históricos.
5. **resmon.exe (*Resource Monitor*):** Analizador en tiempo real del uso granular de CPU, memoria física, actividad de disco por archivo y tráfico de red por conexión.
6. **typeperf.exe (*Performance Counter Output*):** Vuelca el valor instantáneo de contadores de rendimiento hacia la consola o archivos delimitados por comas (CSV).
7. **logman.exe (*Performance Log Manager*):** Administra y programa recolectores de seguimiento de eventos de Windows (ETW) y registros de rendimiento.
8. **wmic.exe (*WMI Command-Line*):** Interfaz de administración basada en Instrumental de Administración de Windows (WMI) para consultar hardware, software y estado operativo del sistema.
9. **systeminfo.exe (*System Information Tool*):** Muestra un resumen técnico detallado de la configuración del sistema operativo, versión del núcleo, parches instalados, memoria física y procesadores.
10. **driverquery.exe (*Device Driver Query*):** Inspecciona y genera una lista de todos los controladores de dispositivo instalados y su estado operativo actual.

5.4.4. Mantenimiento, imágenes, arranque y energía

1. **sfc.exe (*System File Checker*):** Examina la integridad criptográfica de todos los archivos protegidos del sistema operativo y restituye versiones dañadas desde el almacén de componentes WinSxS.
2. **dism.exe (*Deployment Image Servicing and Management*):** Herramienta medular de mantenimiento y reparación de imágenes de Windows (WIM, VHD), habilitación de características opcionales y saneamiento del almacén de componentes.
3. **bcdedit.exe (*Boot Configuration Data Editor*):** Administra y edita el almacén de datos de configuración de arranque (BCD) de Windows NT, configurando parámetros del cargador de arranque (*Windows Boot Manager*).
4. **bootrec.exe (*Boot Recovery Tool*):** Utilería del entorno de recuperación para reparar el registro de arranque maestro (MBR), el sector de arranque de partición y reconstruir el almacén BCD.
5. **reagentc.exe (*Recovery Agent Configurator*):** Configura y activa la imagen del entorno de recuperación de Windows (Windows RE) para labores de diagnóstico automatizado.
6. **shutdown.exe (*Shutdown / Restart Utility*):** Ejecuta de forma controlada el apagado, reinicio, cierre de sesión o hibernación del equipo local o remoto, con soporte de temporizadores.
7. **powercfg.exe (*Power Configuration Tool*):** Configura de manera exhaustiva planes de administración de energía, estados de suspensión (S1–S4) y genera reportes analíticos de consumo y batería.
8. **msinfo32.exe (*System Information*):** Interfaz gráfica integral que reúne diagnósticos de recursos de hardware, conflictos de E/S, puertos de memoria y controladores de software.
9. **sysprep.exe (*System Preparation Tool*):** Prepara una instalación de Windows para su captura y clonación masiva eliminando identificadores de seguridad (SID) y controladores específicos de máquina.
10. **wusa.exe (*Windows Update Standalone Installer*):** Ejecutable encargado de instalar actualizaciones y revisiones de seguridad contenidas en paquetes autónomos `.msu`.
11. **bitsadmin.exe (*BITS Administration Utility*):** Administra tareas del servicio de transferencia inteligente en segundo plano (BITS) para descargas asíncronas optimizadas de archivos.

5.4.5. Seguridad, control de acceso y criptografía

1. **whoami.exe (*Who Am I Tool*):** Proyecta la identidad del usuario actual, su identificador de seguridad (SID), grupos de pertenencia y privilegios asignados al token de acceso.
2. **icacls.exe (*Integrity & Access Control Lists*):** Modifica, respalda y restaura listas de control de acceso discrecional (DACL) e integridad en archivos y carpetas del sistema NTFS.
3. **takeown.exe (*Take Ownership Utility*):** Restituye la propiedad de un archivo o directorio a un usuario o al grupo de administradores locales.
4. **gpupdate.exe (*Group Policy Update*):** Fuerza la actualización y aplicación inmediata de las políticas de grupo locales y de dominio de Active Directory.
5. **gpresult.exe (*Group Policy Result*):** Diagnostica y despliega el conjunto resultante de directivas (RSoP) aplicadas al equipo o al usuario en sesión.
6. **certutil.exe (*Certificate Utility*):** Herramienta criptográfica versátil para verificar certificados digitales, gestionar autoridades de certificación y calcular resúmenes criptográfi-

cos (SHA-256, MD5).
7. **klist.exe (*Kerberos Ticket List*):** Inspecciona, purga y renueva la caché de boletos Kerberos (boletos de concesión TGT y boletos de servicio) en sesiones autenticadas.
8. **auditpol.exe (*Audit Policy Utility*):** Configura y consulta de forma granular las directivas y subcategorías de auditoría de seguridad del sistema.
9. **manage-bde.exe (*BitLocker Drive Encryption Tool*):** Administra la protección de cifrado de volumen BitLocker, permitiendo bloquear, desbloquear y gestionar protectores de clave y módulo TPM.

5.4.6. Registro de Windows, servicios y motores de script

1. **reg.exe (*Registry Console Tool*):** Permite consultar, agregar, modificar, exportar e importar claves y valores en el Registro de Windows por línea de comandos.
2. **regedit.exe (*Registry Editor*):** Interfaz gráfica interactiva para la exploración y edición jerárquica de las colmenas del Registro del sistema.
3. **sc.exe (*Service Controller*):** Se comunica de forma directa con el Service Control Manager para crear, reconfigurar parámetros, pausar y eliminar servicios de Windows.
4. **regsvr32.exe (*Register Server*):** Registra y anula el registro de componentes de software OLE y controles COM en bibliotecas dinámicas DLL.
5. **rundll32.exe (*Run DLL Entry Point*):** Carga y ejecuta puntos de entrada y funciones exportadas de bibliotecas de enlace dinámico de 32 o 64 bits.
6. **msconfig.exe (*System Configuration*):** Herramienta visual para parametrizar el inicio selectivo del sistema, arranque a prueba de fallos y depuración de servicios.
7. **wevtutil.exe (*Windows Event Log Utility*):** Consulta, exporta, archiva y depura registros de eventos del sistema estructurados en formato Evtx.
8. **cmd.exe (*Windows Command Interpreter*):** Intérprete nativo de comandos de consola de la arquitectura Windows NT.
9. **powershell.exe (*Windows PowerShell Host*):** Entorno de automatización y shell de línea de comandos basada en objetos sobre la plataforma .NET.
10. **cscript.exe (*Windows Script Host - Console*):** Motor de ejecución de guiones en VBScript y JScript diseñado para ejecutarse dentro de la consola de texto.
11. **wscript.exe (*Windows Script Host - Windows*):** Motor de ejecución de guiones en VBScript y JScript provisto de cuadros de diálogo y ventanas gráficas.

5.4.7. Filtrado de texto, datos y utilerías de soporte

1. **find.exe (*Find Text String*):** Busca patrones de cadenas literales de caracteres dentro de archivos de texto o flujos redirigidos.
2. **findstr.exe (*Find String with Regular Expressions*):** Búsqueda avanzada de patrones de texto utilizando expresiones regulares estándar.
3. **sort.exe (*Sort Utility*):** Ordena alfabética o numéricamente las líneas de texto procedentes de un archivo o del flujo de entrada estándar.
4. **fc.exe (*File Compare*):** Compara dos archivos de texto línea por línea o dos archivos binarios byte a byte, reportando las diferencias encontradas.
5. **comp.exe (*Compare Utility*):** Compara de manera rápida el contenido de pares o grupos de archivos independientes.
6. **clip.exe (*Clipboard Redirector*):** Redirige la salida de texto de cualquier orden de consola directamente hacia el portapapeles del sistema operativo.

7. **where.exe (*Where Is Utility*):** Localiza y muestra la ruta de los ejecutables que coinciden con el patrón especificado dentro de las carpetas de la variable PATH.
8. **doskey.exe (*Doskey Utility*):** Permite recuperar el historial de comandos ejecutados, editar líneas de órdenes y configurar macros de consola.
9. **timeout.exe (*Timeout Delay Tool*):** Suspende la ejecución del intérprete de comandos por un número determinado de segundos o hasta que se presione una tecla.

5.4.8. Consolas de gestión Microsoft (MMC y complementos .msc)

El marco de gestión extensible Microsoft Management Console (`mmc.exe`) aloja diversas herramientas administrativas en formato `.msc` ubicadas directamente en `System32`:

1. **mmc.exe (*Microsoft Management Console*):** Proceso anfitrión principal que proporciona un entorno integrado para hospedar complementos de administración gráfica (*snap-ins*).
2. **compmgmt.msc (*Computer Management*):** Consola integral que unifica la administración de discos, visor de eventos, carpetas compartidas y servicios locales.
3. **devmgmt.msc (*Device Manager*):** Inspección gráfica, actualización de controladores y resolución de conflictos de dispositivos de hardware.
4. **diskmgmt.msc (*Disk Management*):** Administración visual de unidades de disco, formateo, creación de volúmenes simples o distribuidos y asignación de letras.
5. **services.msc (*Services Management*):** Interfaz visual para iniciar, pausar, detener y parametrizar los servicios en segundo plano del sistema.
6. **eventvwr.msc (*Event Viewer*):** Visor estructurado de registros de eventos agrupados en registros de aplicaciones, seguridad, instalación y sistema.
7. **secpol.msc (*Local Security Policy*):** Configuración de directivas de auditoría, asignación de derechos de usuario y directivas de contraseñas locales.
8. **gpedit.msc (*Local Group Policy Editor*):** Editor jerárquico de políticas de grupo que permite configurar miles de parámetros del sistema operativo y del usuario.
9. **certmgr.msc (*Certificates Manager*):** Administración visual de certificados digitales y almacenes de entidades de confianza para el usuario local.
10. **lusrmgr.msc (*Local Users and Groups*):** Administración gráfica de cuentas de usuario locales y pertenencia a grupos de seguridad del equipo.

6. Administración de recursos y comandos

6.1. Administración de memoria

La administración de la memoria principal (RAM) constituye una de las responsabilidades más complejas y determinantes del sistema operativo, condicionando directamente el nivel de multiprogramación, el rendimiento del procesador y la estabilidad del software en ejecución (Stallings, 2018, §7.1, p. 314). La memoria principal es un recurso finito y volátil que almacena tanto las instrucciones binarias del núcleo como los espacios de direcciones pertenecientes a los procesos de usuario. Para garantizar una operación eficiente y segura, el subsistema de gestión de memoria debe satisfacer cinco requerimientos fundamentales (Stallings, 2018, §7.1, pp. 315–318):

1. **Reubicación (*Relocation*):** El procesador y el sistema operativo no pueden predecir con antelación en qué región física de la RAM se cargará un proceso cuando sea admitido

o conmutado desde el disco. En consecuencia, las referencias a memoria deben expresarse como direcciones lógicas relativas que se traducen dinámicamente a direcciones físicas durante la fase de ejecución.

2. **Protección (*Protection*):** Cada proceso debe estar blindado contra accesos no autorizados procedentes de otros procesos, impidiendo lecturas o escrituras ilícitas tanto en el espacio del núcleo como en los segmentos de datos ajenos. La validación de los límites de memoria debe resolverse en tiempo de ejecución por el hardware de la unidad de gestión de memoria (MMU, *Memory Management Unit*).

3. **Compartición (*Sharing*):** Aunque la protección exige aislamiento, el sistema debe proporcionar mecanismos controlados que permitan a múltiples procesos compartir de forma concurrente zonas comunes de memoria, tales como bibliotecas dinámicas compartidas de solo lectura o regiones de memoria compartida para comunicación interproceso (IPC).

4. **Organización lógica (*Logical Organization*):** La memoria lineal y no estructurada del hardware contrasta con la concepción del software, el cual se organiza en módulos, procedimientos y estructuras de datos independientes. El sistema operativo proporciona mecanismos (como la segmentación) que reflejan esta estructura modular.

5. **Organización física (*Physical Organization*):** La computadora estructura el almacenamiento en una jerarquía de velocidad y costo (registros, memorias caché L1/L2/L3, memoria principal RAM y almacenamiento secundario). El administrador de memoria se encarga de regular el flujo bidireccional de datos entre la memoria volátil rápida y el almacenamiento persistente masivo.

6.1.1. Esquemas de asignación contigua y fragmentación

En los primeros desarrollos de la multiprogramación se implementaron esquemas de asignación contigua de memoria, donde cada proceso ocupa un bloque único e ininterrumpido de direcciones físicas (Silberschatz et al., 2006, §8.3, pp. 280–285):

- *Particionamiento estático (fijo):* La memoria principal se divide al momento de inicialización del sistema en particiones de tamaño predeterminado (iguales o variables). Cada partición aloja exactamente un proceso. Este método adolece de una severa **fragmentación interna**, producida cuando el proceso asignado requiere menos espacio que el tamaño total del bloque, desperdiciando la porción sobrante en su interior.

- *Particionamiento dinámico (variable):* Las particiones se crean de forma dinámica con el tamaño exacto solicitado por cada proceso que ingresa a la memoria. Aunque este esquema elimina la fragmentación interna, genera con el tiempo una acusada **fragmentación externa**: a medida que los procesos terminan y abandonan la memoria, el espacio libre queda disperso en multitud de pequeños huecos no contiguos en los que no cabe ningún proceso nuevo, a pesar de que la suma total de memoria desocupada sea suficiente. Para mitigarla se emplean estrategias de asignación como el primer ajuste (*first-fit*), mejor ajuste (*best-fit*) y peor ajuste (*worst-fit*), complementadas periódicamente con técnicas de **compactación**, consistentes en desplazar los procesos activos hacia un extremo de la memoria física para unificar los huecos libres en un único bloque masivo, a expensas de un elevado consumo de ciclos de reloj de la CPU.

6.1.2. Paginación pura, segmentación y memoria virtual

Para superar definitivamente las restricciones de la fragmentación externa y eliminar la exigencia de contigüidad física, los sistemas operativos modernos adoptaron esquemas no contiguos basados en hardware (Stallings, 2018, §7.3–7.4, pp. 328–345):

- **Paginación (*Paging*):** El espacio de memoria física se divide en bloques de tamaño fijo denominados *marcos de página* (*page frames*), mientras que el espacio de direcciones lógicas del proceso se particiona en bloques del mismo tamaño llamados *páginas* (*pages*), típicamente de 4 KB en arquitecturas x86 y ARM. La MMU utiliza una **tabla de páginas** (*page table*) para mapear cada página lógica a un marco físico cualquiera en memoria, garantizando la ausencia total de fragmentación externa y restringiendo la fragmentación interna a únicamente la última página del proceso. Para acelerar la traducción de direcciones virtuales a físicas sin consultar la RAM en cada instrucción, se implementa una memoria asociativa de alta velocidad denominada búfer de traducción anticipada (TLB, *Translation Lookaside Buffer*) (Silberschatz et al., 2006, §8.4, pp. 288–293).
- **Segmentación (*Segmentation*):** Divide el espacio de memoria lógica en bloques de longitud variable denominados segmentos, correspondientes a divisiones lógicas del programa (código, datos globales, pila, funciones). Cada dirección se compone de un par ordenado `<segmento, desplazamiento>` gestionado mediante una tabla de segmentos que incluye la dirección base y el límite de protección.

La culminación de estas técnicas es la **memoria virtual**, una arquitectura en la que el espacio de direcciones virtuales accesible por los procesos es sustancialmente mayor que la memoria física real instalada en el equipo (Stallings, 2018, §8.1, pp. 358–363). Su materialización se sustenta en la **paginación por demanda** (*demand paging*), donde las páginas del programa residen inicialmente en almacenamiento secundario (archivo de paginación o partición de intercambio *swap*) y solo se transfieren a la memoria RAM cuando el hilo en ejecución intenta acceder a ellas.

Cuando el procesador referencia una dirección lógica cuya página no está cargada físicamente en RAM (indicado por un bit de presencia/validez en cero en la tabla de páginas), la MMU genera una interrupción por **fallo de página** (*page fault*) (Silberschatz et al., 2006, §9.2, pp. 318–324). El núcleo suspende el proceso afectado, localiza un marco libre en memoria física (o expulsa una página residente si no hay marcos disponibles), lee la página requerida desde el disco mediante operaciones asíncronas de E/S, actualiza las entradas correspondientes en la tabla de páginas y reanuda la instrucción interrumpida con total transparencia para el software.

6.1.3. Políticas de reemplazo de páginas e hiperpaginación

Si al producirse un fallo de página la memoria principal se encuentra saturada, el sistema debe seleccionar qué marco desalojar aplicando un algoritmo de reemplazo de páginas (Stallings, 2018, §8.2, pp. 372–382):

- *Algoritmo óptimo de Belady (OPT):* Reemplaza la página que no será referenciada durante el período más largo en el futuro. Es teóricamente perfecto pero inalcanzable en la práctica, sirviendo únicamente como cota de referencia comparativa.
- *FIFO (First-In, First-Out):* Desaloja la página más antigua en memoria. Es sencillo de implementar pero puede incurrir en la anomalía de Belady, donde aumentar el número de

marcos físicos de memoria asignados produce, paradójicamente, un mayor número de fallos de página.

- *LRU (Least Recently Used):* Expulsa la página que no ha sido utilizada durante el intervalo temporal más prolongado, apoyándose en el principio de cercanía de referencias. Aunque no presenta la anomalía de Belady, su implementación pura requiere marcas de tiempo o pilas por hardware costosas.
- *Algoritmo del reloj (Clock / Segunda Oportunidad):* Aproximación eficiente a LRU utilizada en sistemas de producción. Mantiene los marcos en una lista circular con un puntero móvil similar a la manecilla de un reloj y un bit de referencia. Si el bit es cero, la página es desalojada; si es uno, el algoritmo pone el bit en cero y avanza al siguiente marco, concediéndole una segunda oportunidad.

Cuando el conjunto de procesos en ejecución carece de marcos suficientes para mantener sus páginas activas, la tasa de fallos de página se incrementa de forma exponencial, provocando que la CPU dedique la totalidad de su tiempo a la transferencia continua de páginas entre la RAM y el disco en lugar de ejecutar instrucciones útiles. Esta degradación catastrófica del rendimiento se denomina **hiperpaginación** (*thrashing*) (Stallings, 2018, §8.2, pp. 384–387). Para prevenirla, el sistema operativo monitorea el modelo del conjunto de trabajo (*working set model*) propuesto por Peter Denning, el cual rastrea las páginas referenciadas por cada proceso en una ventana de tiempo reciente Δ, reduciendo el grado de multiprogramación si la demanda combinada excede la capacidad física instalada.

6.2. Administración de procesos

Un proceso representa la abstracción computacional primordial de un sistema operativo, formalizado como una unidad de trabajo activa constituida por un programa en ejecución, sus recursos asignados y su estado dinámico en el procesador (Silberschatz et al., 2006, §3.1, p. 82). El espacio de memoria virtual asignado a un proceso se estructura internamente en cuatro segmentos bien definidos:

- *Segmento de texto o código:* Contiene las instrucciones binarias legibles por la CPU, marcado como de solo lectura para evitar corrupciones y permitir su compartición concurrente entre instancias.
- *Segmento de datos:* Alberga las variables globales y estáticas del programa, subdividido en datos inicializados y el segmento BSS (*Block Started by Symbol*) para datos no inicializados.
- *Montículo (Heap):* Región de asignación dinámica de memoria en tiempo de ejecución, gestionada por llamadas del programador (`malloc`, `free`, `new`) y que crece hacia direcciones de memoria más altas.
- *Pila (Stack):* Estructura LIFO que almacena los registros de activación de funciones, variables locales y direcciones de retorno, creciendo típicamente en sentido inverso hacia direcciones más bajas.

6.2.1. Bloque de Control del Proceso (PCB) y modelo de estados

Para gobernar el ciclo de vida de los procesos, el núcleo mantiene para cada uno de ellos una estructura de datos maestra denominada Bloque de Control del Proceso (PCB,

Process Control Block) o descriptor de proceso (Stallings, 2018, §3.2, pp. 115–121). El PCB condensa la totalidad de la información operativa del proceso:

- *Identificador del proceso (PID) y credenciales:* Identificador único numérico, PID del proceso padre (PPID) y credenciales de usuario y grupo (UID, GID) que determinan los privilegios del token de acceso.
- *Estado del proceso:* Condición actual en la que se encuentra el proceso dentro del ciclo de vida del planificador.
- *Contexto del procesador:* Valores guardados del contador de programa (PC), registros de propósito general, puntero de pila (SP) y palabras de estado del procesador (PSW), indispensables para restaurar la ejecución tras una conmutación de contexto (*context switch*).
- *Información de planificación:* Prioridad numérica del proceso, punteros a colas de espera, cuota de CPU acumulada y métricas de envejecimiento.
- *Descriptores de memoria:* Punteros a las tablas de páginas, límites de segmentos y mapas de memoria virtual asignada.
- *Estado de contabilidad y E/S:* Descriptores de archivos abiertos, dispositivos asignados, sockets de red y estadísticas de consumo de recursos.

El tránsito de un proceso se formaliza mediante el modelo de estados (Stallings, 2018, §3.2, pp. 110–115). En su arquitectura estándar de cinco estados, el proceso inicia en estado **Nuevo** (*New*) al solicitarse su creación; pasa a **Listo** (*Ready*) una vez admitido y alojado en memoria en espera de asignación de CPU; cuando el despachador (*dispatcher*) le concede el procesador, entra en **En ejecución** (*Running*); si debe esperar la resolución de un evento asíncrono o una operación de E/S, pasa al estado **Bloqueado** o en espera (*Waiting/Blocked*); y al completar su secuencia de instrucciones transita a **Terminado** (*Terminated*), donde permanece transitoriamente como proceso difunto (*zombie*) hasta que su proceso padre lee su código de terminación mediante la llamada del sistema `wait`. Asimismo, para gestionar la saturación de memoria RAM, los sistemas avanzados incorporan los estados suspendidos (**Listo/Suspendido** y **Bloqueado/Suspendido**), transfiriendo el mapa del proceso hacia almacenamiento secundario.

6.2.2. Hilos de ejecución y planificación de CPU

Los sistemas modernos distinguen de forma explícita entre la unidad de asignación de recursos (el proceso) y la unidad de despacho y ejecución sobre el procesador, denominada **hilo de ejecución** (*thread* o hebra) (Stallings, 2018, §4.1, pp. 158–165). Los hilos que comparten un mismo proceso conviven en el mismo espacio de direcciones virtuales y comparten descriptores de archivos, pero poseen su propio contador de programa, registros y pila individual. Esto reduce drásticamente el costo temporal de la conmutación de contexto en comparación con los procesos pesados. Atendiendo al nivel de gestión, se dividen en hilos a nivel de usuario (ULT, gestionados por bibliotecas en espacio de usuario sin conocimiento del kernel) e hilos a nivel de núcleo (KLT, gestionados de forma nativa por el planificador del sistema operativo), adoptando modelos de asignación 1:1, N:1 o M:N (Silberschatz et al., 2006, §4.2, pp. 120–128).

La asignación óptima de la CPU entre los hilos listos para ejecución es competencia del planificador de corto plazo (*short-term scheduler* o *dispatcher*), el cual puede operar bajo modalidades no apropiativas (el hilo conserva la CPU hasta que termina voluntariamente o emite una llamada bloqueante) o apropiativas (el núcleo desaloja el hilo mediante una interrupción de reloj si agota su cuanto de tiempo o aparece un hilo de mayor prioridad)

(Stallings, 2018, §9.2, pp. 418–422). Entre los algoritmos clásicos de planificación destacan:

- **FCFS (*First-Come, First-Served*):** No apropiativo. Asigna la CPU en estricto orden de llegada. Simple de implementar mediante colas FIFO, pero susceptible al efecto convoy, donde procesos cortos se ven postergados por procesos largos de intensiva computación.
- **SJF (*Shortest Job First*) y SRTF (*Shortest Remaining Time First*):** Asignan la CPU al proceso cuya ráfaga de CPU sea más corta (SJF no apropiativo, SRTF apropiativo). Minimizan el tiempo de espera promedio, pero son difíciles de implementar en sistemas de propósito general al exigir predecir la duración de las ráfagas, corriendo el riesgo de provocar inanición (*starvation*) en trabajos largos.
- **Turno Rotatorio (*Round Robin*, RR):** Apropiativo. Diseñado para sistemas de tiempo compartido. Cada proceso recibe un cuanto de tiempo fijo (*time quantum*); al expirar, se genera una interrupción de reloj y el proceso pasa al final de la cola de listos, logrando una excelente respuesta interactiva.
- **Colas Multinivel con Retroalimentación (MLFQ, *Multi-Level Feedback Queues*):** Estructura procesos en múltiples colas con diferentes niveles de prioridad y cuantos de tiempo. Los procesos que agotan su cuanto descienden a colas de menor prioridad, mientras que los procesos de intensiva E/S permanecen en colas superiores, adaptándose dinámicamente al comportamiento de la carga.

6.2.3. Sincronización y comunicación interproceso (IPC)

La ejecución concurrente de hilos y procesos en memoria compartida o multiprocesamiento introduce el riesgo de condiciones de carrera (*race conditions*), situaciones donde el resultado final de la computación depende del orden arbitrario de ejecución de los hilos (Silberschatz et al., 2006, §6.1, pp. 191–194). Para salvaguardar la consistencia, el sistema operativo implementa soluciones al problema de la sección crítica, satisfaciendo exclusión mutua, progreso y espera limitada mediante primitivas de sincronización:

- *Cerrojos de exclusión mutua (Mutex):* Mecanismos binarios de bloqueo de adquisición exclusiva.
- *Semáforos de Dijkstra:* Variables atómicas gestionadas mediante primitivas `wait()` (P) y `signal()` (V), empleadas en exclusión mutua y en esquemas de sincronización.
- *Monitores:* Estructuras de sincronización de alto nivel a nivel de lenguaje con variables de condición.
- *Mecanismos de IPC:* Comunicación mediante paso de mensajes a través de tuberías anónimas o con nombre (*pipes* o tuberías FIFO), memoria compartida (*shared memory*) y sockets de red de dominio UNIX o TCP/IP (Silberschatz et al., 2006, §3.5, pp. 98–108).

6.3. Comandos de sistemas operativos

El intérprete de órdenes o interfaz de línea de comandos (CLI, *Command Line Interface*) representa la herramienta medular para la administración avanzada, automatización y control operativo de un sistema informático. En la computación contemporánea coexisten dos paradigmas dominantes:

1. **El ecosistema POSIX / Linux:** Sustentado en la filosofía UNIX de utilidades pequeñas, modulares y reutilizables (conjuntos de herramientas como GNU Coreutils, procps-ng, util-linux e iproute2), interconectadas mediante flujos de texto plano (*pipelines*) a través de descriptores estándar (`stdin`, `stdout`, `stderr`) (Stallings, 2018, §1.2, p. 3).

2. **El ecosistema Microsoft Windows:** Compuesto por el intérprete clásico de órdenes en modo texto (`cmd.exe`) y el entorno moderno de automatización PowerShell, el cual sustituye la transmisión de cadenas de caracteres planas por tuberías orientadas a objetos estructurados sobre la plataforma .NET (Deitel et al., 2004, §21.2, p. 1039).

A continuación se presenta un catálogo técnico sistemático que categoriza y define rigurosamente los comandos y utilidades indispensables en ambos entornos operativos, agrupados por áreas funcionales de administración:

6.3.1. Navegación y manipulación de archivos y directorios

- `ls` (Linux) / `dir` (Windows): Lista el contenido de directorios, atributos de archivo, marcas temporales y permisos.
- `cd` (Linux/Windows): Modifica el directorio de trabajo activo del proceso en ejecución dentro del árbol del sistema de archivos.
- `pwd` (Linux) / `cd` (sin argumentos en Windows): Muestra la ruta absoluta canónica del directorio de trabajo actual.
- `mkdir` (Linux) / `md` (Windows): Crea uno o varios directorios nuevos en la ruta especificada.
- `rmdir` (Linux) / `rd` (Windows): Elimina directorios existentes, requiriendo parámetros recursivos si contienen archivos.
- `cp` (Linux) / `copy` o `xcopy` (Windows): Duplica archivos y estructuras de directorios hacia una ubicación de destino.
- `mv` (Linux) / `move` (Windows): Desplaza o renombra archivos y carpetas dentro de la jerarquía del sistema de archivos.
- `rm` (Linux) / `del` o `erase` (Windows): Suprime archivos del almacenamiento liberando sus inodos o registros de metadatos.
- `touch` (Linux): Actualiza las marcas temporales de acceso y modificación de un archivo o lo crea si no existe.
- `cat` (Linux) / `type` (Windows): Concatena y proyecta el contenido de archivos de texto en la salida estándar.
- `ln` (Linux) / `mklink` (Windows): Crea enlaces duros (*hard links*) o enlaces simbólicos blandos (*symlinks*).

6.3.2. Gestión, supervisión y control de procesos

- `ps` (Linux) / `tasklist` (Windows): Informa sobre los procesos activos en ejecución, identificadores PID, sesiones y consumo de memoria.
- `top` o `htop` (Linux) / `resmon` (Windows): Monitor interactivo en tiempo real de consumo de CPU, memoria, hilos y carga del sistema.
- `kill` (Linux) / `taskkill` (Windows): Envía señales de terminación forzada (`SIGKILL` / `/F`) o grácil (`SIGTERM`) a procesos.
- `killall` (Linux) / `Stop-Process` (PowerShell): Finaliza todos los procesos que coincidan con un nombre de imagen binaria.

- `nice` y `renice` (Linux): Asigna y modifica la prioridad de planificación estática (*niceness*) de un proceso ante el kernel.
- `free` (Linux): Informa sobre la cantidad de memoria física RAM y memoria de intercambio (*swap*) libre y ocupada.
- `uptime` (Linux): Indica el tiempo transcurrido desde el último encendido del sistema y el promedio de carga en 1, 5 y 15 minutos.
- `vmstat` (Linux): Proyecta estadísticas globales de memoria virtual, procesos bloqueados, paginación, bloques de disco y actividad de CPU.

6.3.3. Redes, conectividad y telecomunicaciones

- `ip` (Linux) / `ipconfig` (Windows): Configura y visualiza direcciones IP, interfaces de red, rutas de enrutamiento y adaptadores.
- `ping` (Linux/Windows): Verifica la conectividad bidireccional y la latencia con un host remoto emitiendo paquetes de eco ICMP.
- `traceroute` (Linux) / `tracert` (Windows): Identifica cada uno de los saltos de enrutamiento intermedios hacia un destino de red.
- `netstat` o `ss` (Linux/Windows): Inspecciona sockets abiertos, puertos TCP/UDP en escucha y tablas de conexión activas.
- `nslookup` o `dig` (Linux/Windows): Consulta de forma interactiva registros en servidores de nombres de dominio (DNS).
- `curl` o `wget` (Linux) / `curl.exe` (Windows): Transfiere datos y descarga recursos a través de protocolos de red (HTTP, HTTPS, FTP).
- `ssh` (Linux/Windows): Establece conexiones remotas seguras cifradas de terminal mediante el protocolo Secure Shell.
- `scp` (Linux/Windows): Transfiere archivos de forma segura entre hosts remotos utilizando autenticación SSH.

6.3.4. Permisos, usuarios y control de acceso

- `chmod` (Linux) / `icacls` (Windows): Modifica los permisos de acceso (lectura, escritura, ejecución en POSIX; listas DACL en NTFS).
- `chown` y `chgrp` (Linux) / `takeown` (Windows): Reasigna el usuario y grupo propietario de archivos y carpetas.
- `useradd` o `adduser` (Linux) / `net user` (Windows): Registra nuevas cuentas de usuario en la base de datos de seguridad del sistema.
- `usermod` (Linux): Modifica parámetros de cuentas de usuario existentes (grupos secundarios, directorio personal, shell).
- `userdel` (Linux) / `net user [usuario] /delete` (Windows): Elimina cuentas de usuario del sistema operativo.
- `passwd` (Linux): Modifica la contraseña de autenticación criptográfica de una cuenta de usuario local.
- `sudo` (Linux): Ejecuta comandos con privilegios elevados de superusuario (*root*) de acuerdo con las directivas de `/etc/sudoers`.
- `su` (Linux): Conmuta la identidad de la sesión activa hacia otro usuario del sistema.
- `id` y `whoami` (Linux/Windows): Muestra el identificador numérico (UID/GID o SID) y el nombre de usuario activo.

6.3.5. Búsqueda, filtrado y procesamiento de flujos de texto

- `grep` (Linux) / `findstr` (Windows): Filtra líneas de texto que coinciden con patrones y expresiones regulares.
- `find` (Linux) / `where` (Windows): Busca archivos y directorios en el árbol del sistema según nombre, tamaño o fecha.
- `sed` (Linux): Editor de flujo no interactivo para transformación, sustitución y manipulación automatizada de texto.
- `awk` (Linux): Lenguaje y procesador de patrones orientado al procesamiento estructurado de texto delimitado por columnas.
- `sort` (Linux/Windows): Ordena alfabética o numéricamente líneas de texto recibidas de archivos o tuberías.
- `uniq` (Linux): Detecta y omite líneas de texto duplicadas contiguas en un flujo ordenado.
- `wc` (Linux) / `Measure-Object` (PowerShell): Cuantifica el número de líneas, palabras y bytes contenidos en un archivo o flujo.
- `cut` (Linux): Extrae campos o secciones delimitadas de cada línea de un flujo de texto.
- `tee` (Linux/PowerShell): Bifurca el flujo de datos, proyectándolo en pantalla mientras lo escribe simultáneamente en un archivo.
- `xargs` (Linux): Construye y ejecuta líneas de órdenes a partir de elementos recibidos desde la entrada estándar.

6.3.6. Almacenamiento, sistemas de archivos y montaje

- `df` (Linux) / `Get-Volume` (PowerShell): Informa sobre el espacio ocupado y disponible en los sistemas de archivos montados.
- `du` (Linux): Calcula y muestra el espacio de disco ocupado por directorios y archivos específicos.
- `mount` (Linux) / `mountvol` (Windows): Acopla un sistema de archivos físico o virtual en el árbol de directorios del sistema.
- `umount` (Linux): Desacopla un sistema de archivos montado previamente, garantizando la escritura de búferes pendientes.
- `fdisk` o `parted` (Linux) / `diskpart` (Windows): Crea, modifica y elimina tablas de particionado MBR y GPT.
- `mkfs` (Linux) / `format` (Windows): Inicializa la estructura lógica de un sistema de archivos (ext4, NTFS, FAT32) en una partición.
- `fsck` (Linux) / `chkdsk` (Windows): Verifica y repara la consistencia estructural de un sistema de archivos.
- `lsblk` (Linux): Despliega en forma de árbol los dispositivos de almacenamiento en bloque conectados y sus particiones.

6.3.7. Administración del sistema, servicios y control de energía

- `systemctl` (Linux) / `sc` (Windows): Administra el ciclo de vida de los servicios del sistema operativo (iniciar, detener, habilitar).
- `journalctl` (Linux) / `wevtutil` (Windows): Consulta y filtra los registros del sistema estructurados generados por el kernel y servicios.
- `shutdown` (Linux/Windows): Programa el apagado ordenado, reinicio o corte de energía de la máquina física.

- `reboot` (Linux) / `shutdown /r` (Windows): Reinicia el sistema operativo de forma controlada.
- `uname` (Linux) / `systeminfo` (Windows): Reporta información detallada sobre la arquitectura del hardware, nombre del núcleo y versión del sistema operativo.
- `dmesg` (Linux): Imprime los mensajes del búfer de anillo del núcleo (*kernel ring buffer*) emitidos durante el arranque y detección de hardware.

7. Conclusiones personales

A lo largo de este trabajo he podido profundizar en los fundamentos teóricos y prácticos que dan vida a los sistemas operativos modernos. Estudiar su evolución histórica me permitió comprender que ninguna característica actual —desde el tiempo compartido hasta la virtualización contemporánea— surgió de forma fortuita, sino como una respuesta directa a los límites físicos del hardware y a la necesidad de maximizar el aprovechamiento de recursos. Esta perspectiva me ayudó a dejar de ver al sistema operativo como una simple capa intermedia invisible y a reconocerlo como una sofisticada estructura de compromisos arquitectónicos, donde cada mecanismo existe para resolver un conflicto concreto entre velocidad, memoria y aislamiento.

La exploración sistemática de más de un centenar de plataformas me dejó una lección muy clara: en la ingeniería de sistemas no existe una solución universal ni un diseño intrínsecamente superior. Cada sistema operativo es el resultado de priorizar ciertos objetivos a expensas de otros. Me resultó fascinante constatar cómo conviven filosofías tan opuestas como la robustez matemática de los micronúcleos y la pragmática eficiencia de los núcleos monolíticos comerciales. Asimismo, analizar las plataformas orientadas a bajos recursos me demostró que el rendimiento no siempre depende de contar con hardware de última generación, sino de la disciplina en el diseño del software y de la eliminación rigurosa de componentes superfluos.

Profundizar en la virtualización transformó profundamente mi visión sobre cómo se gestiona la infraestructura de cómputo. Me permitió entender que la verdadera potencia de un sistema operativo radica en su capacidad de crear abstracciones tan convincentes que desacoplan por completo el software de la máquina física. Comprender este nivel de aislamiento no solo me aclaró cómo operan los centros de datos modernos y los entornos de nube, sino que me hizo dimensionar la elegancia que supone engañar al procesador para ejecutar múltiples sistemas concurrentes con una penalización de rendimiento prácticamente imperceptible.

En cuanto al ecosistema de Windows, examinar a fondo la estructura de su directorio de sistema me permitió desmitificar procesos que habitualmente observaba en ejecución pero cuyo propósito real desconocía. Descubrir la lógica detrás de la inicialización del sistema y entender por qué persisten ejecutables históricos por razones de compatibilidad me dio una lección invaluable sobre la ingeniería de software en el mundo real. Me quedó claro que mantener la estabilidad de una plataforma que debe soportar décadas de software legado exige un balance extraordinario entre modernización y preservación de convenciones de bajo nivel.

El estudio detallado de la administración de memoria y procesos me confirmó que estos dos pilares constituyen el verdadero corazón de la disciplina. Comprender el mecanismo de memoria virtual y la paginación por demanda me enseñó por qué esta tecnología es una de las mayores hazañas intelectuales de la computación, logrando que los programas

operen sin preocuparse por los límites reales de la memoria física. Del mismo modo, analizar la concurrencia y la sincronización interproceso me hizo valorar lo frágil que resulta la coherencia cuando múltiples hilos compiten por recursos compartidos, reafirmando la necesidad de aplicar mecanismos de exclusión mutua rigurosos para evitar fallos sutiles y catastróficos.

En conclusión, este recorrido integral no solo consolidó mis conocimientos teóricos, sino que transformó mi relación con la interfaz de línea de comandos. Aprender a comunicarme con el sistema operativo sin la intermediación de entornos gráficos me brindó una sensación de control, autonomía y precisión indispensable para mi formación como futuro ingeniero en computación. Considero que dominar cómo piensa y cómo administra sus recursos la máquina es el conocimiento fundamental que diferencia a quien solo utiliza la tecnología de quien es capaz de diseñarla, optimizarla y resolver problemas complejos en su raíz.

Referencias

Carretero Pérez, J., García Carballeira, F., Miguel Anasagasti, P. d., & Pérez Costoya, F. (2001). *Sistemas operativos: una visión aplicada* (1.ª ed.). McGraw-Hill/Interamericana de España.

Deitel, H. M., Deitel, P. J., & Choffnes, D. R. (2004). *Operating systems* (3.ª ed.). Pearson Prentice Hall.

McHoes, A. M., & Flynn, I. M. (2011). *Sistemas operativos* (6.ª ed.). Cengage Learning Editores.

Microsoft Corporation. (2023). *Windows commands reference*. Consultado el 25 de octubre de 2026, desde https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/windows-commands

Microsoft Corporation. (2024). *Windows 11 specs and system requirements*. Consultado el 25 de octubre de 2026, desde https://www.microsoft.com/en-us/windows/windows-11-specifications

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill/Interamericana de España.

Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.) [Global Edition]. Pearson Education Limited.

The Debian Project. (2023). *Debian gnu/linux 12 (bookworm) release notes*. Consultado el 25 de octubre de 2026, desde https://www.debian.org/releases/bookworm/releasenotes

The FreeBSD Project. (2023). *Freebsd 14.0-release release notes*. Consultado el 25 de octubre de 2026, desde https://www.freebsd.org/releases/14.0R/relnotes/